

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM

KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC: LẬP TRÌNH PYTHON

Đề tài:

LẬP TRÌNH GAME TETRIS

GVHD: TS. Phan Thị Thê

SVTH:

- | | |
|---------------------------|----------|
| 1. Nguyễn Vĩnh An | 23162002 |
| 2. Võ Minh Khang | 23162041 |
| 3. Đặng Thiện Quang | 23162078 |
| 4. Nguyễn Phương Hoài Nam | 23162060 |

Mã lớp học: IPPA233277_08

Thành phố Hồ Chí Minh, Tháng 11 năm 2024

ĐH SƯ PHẠM KỸ THUẬT TP.HCM
KHOA CNTT

XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh Phúc

PHIẾU NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Họ và tên Sinh viên 1 : Nguyễn Vĩnh An	MSSV 1: 23162002
Họ và tên Sinh viên 2 : Võ Minh Khang	MSSV 2: 23162041
Họ và tên Sinh viên 3: Đảng Thiện Quang	MSSV 3: 23162078
Họ và tên Sinh viên 4: Nguyễn Phương Hoài Nam	MSSV 4: 23162060
Ngành: An toàn Thông tin	
Tên đề tài: Lập trình game Tetris	
Họ và tên Giáo viên hướng dẫn: TS. Phan Thị Thể	

NHẬN XÉT

1. Nội dung đề tài & khối lượng thực hiện

2. Khuyết điểm

3. Đề nghị cho bảo vệ hay không ?

4. Đánh giá loại :

5. Điểm :

Tp. Hồ Chí Minh, ngày tháng năm 20

Giáo viên hướng dẫn

(Ký & ghi rõ họ tên)

LỜI CẢM ƠN

Kính gửi cô Thê

Chúng em xin gửi lời cảm ơn chân thành đến cô đã tận tình hướng dẫn và hỗ trợ nhóm em trong suốt quá trình thực hiện đồ án cuối kỳ. Những kiến thức quý báu và sự tận tâm của cô đã giúp chúng em vượt qua những khó khăn và hoàn thiện đồ án của mình.

Không chỉ vậy, sự động viên và khích lệ từ cô đã giúp chúng em tự tin hơn, sẵn sàng đối mặt với thử thách và phát triển bản thân. Chúng em rất trân trọng những góp ý, chỉ bảo của cô, điều đó không chỉ giúp nhóm em phát triển kỹ năng chuyên môn mà còn nâng cao khả năng tư duy sáng tạo. Chúng em hy vọng sẽ có nhiều cơ hội học hỏi từ cô hơn nữa trong tương lai.

Một lần nữa, chúng em xin chân thành cảm ơn cô!

Trân trọng,

Nhóm 13

Trường ĐH Sư Phạm Kỹ Thuật TP.HCM

Khoa : CNTT

ĐỀ CƯƠNG BÁO CÁO ĐỒ ÁN CUỐI KÌ

Họ và tên Sinh viên 1 : Nguyễn Vĩnh An	MSSV 1: 23162002
Họ và tên Sinh viên 2 : Võ Minh Khang	MSSV 2: 23162041
Họ và tên Sinh viên 3: Đặng Thiện Quang	MSSV 3: 23162078
Họ và tên Sinh viên 4: Nguyễn Phương Hoài Nam	MSSV 4: 23162060
Thời gian làm báo cáo : từ : 18/10/2024	Đến : 20/11/2024

Chuyên ngành: An toàn Thông tin

Tên luận văn: Lập trình game Tetris

Giáo viên hướng dẫn: Phan Thị Thể

Nhiệm Vụ Của Báo Cáo :

1. Giới thiệu đề tài
2. Khảo sát và phân tích yêu cầu
3. Xây dựng mô hình hệ thống
4. Thiết kế giao diện người dùng
5. Thiết kế cơ sở dữ liệu
6. Phát triển và lập trình hệ thống
7. Kiểm thử hệ thống
8. Triển khai hệ thống
9. Đánh giá kết quả đạt được
10. Kết luận và hướng phát triển
11. Tài liệu tham khảo
12. Phụ lục

KẾ HOẠCH THỰC HIỆN

STT	Thời gian	Công việc	Ghi chú
1	18/10 – 20/10	Phân tích yêu cầu và xác định mục tiêu đồ án	Tìm hiểu đề tài và các yêu cầu kỹ thuật
2	21/10 – 23/10	Nghiên cứu tài liệu và thu thập thông tin	Xác định nguồn tài liệu và dữ liệu
3	24/10 – 26/10	Xây dựng và phân tích mô hình lý thuyết	Phác thảo hướng triển khai đồ án
4	27/10 – 29/10	Thiết kế hệ thống: cơ sở dữ liệu, giao diện, luồng xử lý	Sử dụng các công cụ hỗ trợ như Drawio, Flowchart
5	30/10 – 10/11	Lập trình và triển khai chức năng	Kiểm tra tính năng và sửa lỗi
6	01/11 – 14/11	Kiểm thử sản phẩm	Đảm bảo hệ thống hoạt động đúng yêu cầu, kiểm thử các tính năng
7	15/11 – 20/11	Viết báo cáo và hoàn thiện đồ án	Chuẩn bị nộp lên trang dạy học số UTEx và thuyết trình

Ý kiến giáo viên phản biện
(ký và ghi rõ họ tên)

Người viết đề cương
(ký và ghi rõ họ tên)

BẢNG PHÂN CÔNG VIỆC, ĐÁNH GIÁ

Họ và tên	Công việc	Đánh giá
Nguyễn Vĩnh An 23162002	Phân tích, thiết kế, tiến hành thực hiện hóa ứng dụng (code giao diện chính, hàm hiệu ứng), viết báo cáo word, làm ppt báo cáo.	Hoàn thành 100%
Võ Minh Khang 23162041	Phân tích, thiết kế, tiến hành thực hiện hóa ứng dụng(code hàm main, tạo lưới), viết báo cáo word, làm ppt báo cáo.	Hoàn thành 100%
Đàng Thiện Quang 23162078	Phân tích, thiết kế, tiến hành thực hiện hóa ứng dụng(code hàm phá hủy, class, vào file), viết báo cáo word, làm ppt báo cáo.	Hoàn thành 100%
Nguyễn Phương Hoài Nam 23162060	Phân tích, thiết kế, tiến hành thực hiện hóa ứng dụng(code hàm độ khó, hàm xếp hạng), viết báo cáo word, làm ppt báo cáo.	Hoàn thành 100%

MỤC LỤC

I. PHẦN MỞ ĐẦU	1
1. Lý do chọn đề tài.....	1
2. Mục đích của đề tài.....	1
3. Phương pháp nghiên cứu	2
4. Ứng dụng nhóm thảo luận, chia sẻ	3
II. PHẦN NỘI DUNG.....	3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	3
1.1. Giới thiệu tổng quan về python và sơ lược về đồ án	3
1.1.1. Tổng quan về python.....	3
1.1.2. Sơ lược về đồ án	4
1.2. Giới thiệu và cách sử dụng của các thư viện sử dụng trong đồ án	6
1.2.1. Thư viện pygame.....	7
1.2.2. Thư viện Random	7
1.2.3. Thư viện Math.....	8
1.2.4. Thư viện Pathlib	8
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG	9
2.1. Đặc tả yêu cầu.....	9
2.1.1. Yêu cầu chức năng	9
2.1.2. Yêu cầu phi chức năng	10
2.2. Đặc tả luồng chức năng	11
2.2.1. Luồng chính	11
2.2.2. Luồng phụ	12
CHƯƠNG 3: THIẾT KẾ VÀ PHÁT TRIỂN HỆ THỐNG.....	13
3.1. Thiết kế giao diện	13
3.1.1. Menu chính	14

3.1.2. <i>Giao diện trò chơi</i>	16
3.1.3. <i>Menu settings</i>	18
3.2. Thiết kế chức năng	19
3.2.1. <i>Hàm logic</i>	19
3.2.2. <i>Hàm xử lý giao diện</i>	28
3.3. Quy trình hoạt động	34
3.3.1. <i>Thiết lập cấu hình ban đầu cho ứng dụng</i>	34
3.3.2. <i>Cách thức hoạt động</i>	41
CHƯƠNG 4: ĐÁNH GIÁ QUÁ TRÌNH HOÀN THÀNH ĐỒ ÁN VÀ GIẢI PHÁP NÂNG CAO HIỆU QUẢ CỦA ỨNG DỤNG GAME TETRIS TRONG THỜI GAN TỐI	54
4.1. Đánh giá quá trình hoàn thành đồ án	54
4.2. Giải pháp nhằm nâng cao hiệu quả của ứng dụng game trong thời gian tối	54
III. KẾT LUẬN	56
1. Kết luận	56
2. Thành tựu	56
TÀI LIỆU THAM KHẢO	57

Mục lục hình ảnh

Hình 1: Menu chính	14
Hình 2: Nhập tên.....	16
Hình 3: Vào chơi.....	16
Hình 4: Top rankings	17
Hình 5. Seclect Difficulty	18
Hình 6. Class piece	19
Hình 7. Hàm set_difficulty	20
Hình 8. Hàm create_grid	20
Hình 9. Hàm convert_shape_format.....	21
Hình 10. Hàm valid_space.....	21
Hình 11. Hàm check_lost	22
Hình 12. Hàm get_shape.....	22
Hình 13. Hàm clear_rows	23
Hình 14. Hàm toggle_pause	23
Hình 15. Hàm button_hover	23
Hình 16. Hàm destroy_blocks	24
Hình 17. Hàm destroy_blocks_with_effectmouse	25
Hình 17.1. Hàm destroy_blocks_with_effectmouse	26
Hình 18. Hàm fireworks_effect_pro	26
Hình 19. Hàm fireworks_effect	27
Hình 19.1. Hàm fireworks_effect	28
Hình 20. Hàm draw_text_midle	28
Hình 21. Hàm draw_grid	29
Hình 22. Hàm draw_exit_button.....	29
Hình 23. Hàm draw_next_shape.....	30
Hình 24. Hàm draw_core	30
Hình 25. Hàm draw_pause_message.....	31
Hình 26. Hàm draw_pause_button	31
Hình 27. Hàm draw_window	32
Hình 28. Hàm draw_button	33

Hình 29. Hàm <code>draw_destroy_count</code>	33
Hình 29. Các thư viện	34
Hình 30. Đường dẫn tệp	35
Hình 31. Khởi tạo Pygame.....	35
Hình 32. Khởi tạo thông số toàn cục	35
Hình 33. Tải ảnh nền	36
Hình 34. Khởi tạo hệ thống âm thanh.....	37
Hình 35. Phát nhạc nền	37
Hình 36. Âm thanh của nút bấm	37
Hình 37. Khối S.....	38
Hình 38. Khối Z.....	38
Hình 39. Khối I	39
Hình 40. Khối O.....	39
Hình 41. Khối J	39
Hình 42. Khối L.....	40
Hình 43. Khối T.....	40
Hình 44. Danh sách các Shapes	40
Hình 45. Danh sách màu của các Shapes	41
Hình 46. Hàm <code>main_menu</code>	41
Hình 46.1. Hàm <code>main_menu</code>	41
Hình 46.2. Hàm <code>main_menu</code>	42
Hình 46.3. Hàm <code>main_menu</code>	42
Hình 46.4. Hàm <code>main_menu</code>	43
Hình 47. Hàm <code>enter_player_name</code>	43
Hình 47.1. Hàm <code>enter_player_name</code>	44
Hình 48. Hàm <code>main</code>	45
Hình 48.1. Hàm <code>main</code>	46
Hình 48.2. Hàm <code>main</code>	46
Hình 48.3. Hàm <code>main</code>	47
Hình 48.4. Hàm <code>main</code>	47
Hình 48.5. Hàm <code>main</code>	48

<i>Hình 48.6. Hàm main</i>	49
<i>Hình 48.7. Hàm main</i>	49
<i>Hình 49. Hàm setting_menu</i>	49
<i>Hình 49.1. Hàm setting_menu</i>	50
<i>Hình 49.2. Hàm setting_menu</i>	51
<i>Hình 50. Hàm display_rankings</i>	51
<i>Hình 50.1. Hàm display_rankings</i>	52
<i>Hình 51. Hàm save_score_and_get_rank</i>	52
<i>Hình 51.1. Xử lý lỗi</i>	53
<i>Hình 51.2. Đọc và xác định thứ hạng</i>	53
<i>Hình 51.3. Chuyển đổi và sắp xếp</i>	53
<i>Hàm 51.3. Xác định thứ hạng</i>	53

I. PHẦN MỞ ĐẦU

1. Lý do chọn đề tài

Lý do chúng tôi chọn đề tài "Lập trình ứng dụng game Tetris" là vì trò chơi này vừa đơn giản, vừa đầy thử thách trong việc phát triển phần mềm. Tetris yêu cầu chúng tôi phải xử lý các thuật toán phức tạp liên quan đến di chuyển, xoay khối, phát hiện va chạm và quản lý không gian trò chơi một cách hiệu quả. Điều này giúp chúng tôi rèn luyện khả năng giải quyết các vấn đề lập trình logic, đồng thời nâng cao kỹ năng tối ưu hóa thuật toán và cải thiện hiệu suất ứng dụng.

Bên cạnh đó, Tetris là một trong những game cổ điển nổi tiếng trên thế giới, việc lập trình một phiên bản của trò chơi này sẽ giúp nhóm hiểu rõ hơn về quy trình phát triển game, từ việc thiết kế các tính năng cơ bản cho đến việc tối ưu hóa giao diện và trải nghiệm người dùng. Qua đó, nhóm không chỉ học hỏi được về thiết kế game mà còn có cơ hội áp dụng các kiến thức lập trình cơ bản như cấu trúc dữ liệu, thuật toán, và lập trình hướng đối tượng vào một dự án thực tế.

2. Mục đích của đề tài

Mục đích của đề tài "Lập trình ứng dụng game Tetris" là phát triển một phiên bản Tetris đơn giản nhưng đầy đủ các tính năng cơ bản của trò chơi. Thông qua đề tài này, nhóm mong muốn phát triển kỹ năng lập trình, đặc biệt là trong các lĩnh vực lập trình hướng đối tượng, xử lý sự kiện, quản lý trạng thái game và tối ưu hóa thuật toán.

Đề tài còn nhằm giúp sinh viên hiểu rõ quy trình phát triển game, từ thiết kế các tính năng cơ bản như di chuyển và xoay khối, phát hiện va chạm, tính điểm cho đến tối ưu hóa hiệu suất trò chơi. Đây là cơ hội để cải thiện kỹ năng giao diện người dùng (UI/UX), giúp trò chơi có trải nghiệm thân thiện và dễ sử dụng. Ngoài ra, việc xử lý các thuật toán liên quan đến xếp hình khối, tính toán điểm số và va chạm sẽ giúp sinh viên áp dụng kiến thức về cấu trúc dữ liệu và thuật toán trong lập trình và còn mở ra cơ hội để sinh viên khám phá và thực hành các công nghệ lập trình game cơ bản như xử lý đồ họa, âm thanh và các yếu tố tương tác trong môi trường game, từ đó chuẩn bị cho các dự án game lớn hơn trong tương lai.

3. Phương pháp nghiên cứu

Để thực hiện đề tài "Lập trình ứng dụng game Tetris", nhóm chúng tôi áp dụng phương pháp nghiên cứu kết hợp giữa lý thuyết và thực hành, thông qua các bước nghiên cứu và phát triển phần mềm theo quy trình phần mềm chuẩn. Cụ thể, phương pháp nghiên cứu được chia thành các giai đoạn chính như sau:

Nghiên cứu lý thuyết và phân tích yêu cầu: Đầu tiên, nhóm nghiên cứu các tài liệu, sách và bài viết liên quan đến lập trình game, đặc biệt là các trò chơi cổ điển như Tetris. Việc này giúp nhóm hiểu rõ các nguyên lý cơ bản trong phát triển game, từ cấu trúc dữ liệu đến các thuật toán xử lý va chạm, tối ưu hóa hiệu suất. Đồng thời, nhóm phân tích các yêu cầu chức năng và phi chức năng của game Tetris, xác định các tính năng cơ bản cần có trong game, như di chuyển khối, xoay khối, tính điểm và điều khiển tốc độ.

Thiết kế game: Sau khi nghiên cứu và phân tích yêu cầu, nhóm tiến hành thiết kế hệ thống game, mối quan hệ và các quy trình trong game. Nhóm cũng thiết kế giao diện người dùng (UI) để đảm bảo tính dễ sử dụng và thân thiện, đảm bảo trải nghiệm người chơi mượt mà và trực quan.

Lập trình và triển khai: Sau giai đoạn thiết kế, nhóm bắt đầu lập trình ứng dụng game Tetris bằng ngôn ngữ Python, sử dụng thư viện Pygame để xử lý đồ họa, âm thanh và các tương tác của người chơi. Các khối được di chuyển và xoay theo các quy tắc đặc trưng của game Tetris, đồng thời tính toán điểm số và phát hiện va chạm. Phương pháp lập trình hướng đối tượng (OOP) được áp dụng để tổ chức mã nguồn theo các lớp và đối tượng, đảm bảo tính tái sử dụng và dễ bảo trì.

Kiểm thử và tối ưu hóa: Sau khi hoàn thành việc lập trình, nhóm tiến hành kiểm thử các tính năng của game để phát hiện và sửa lỗi. Các thử nghiệm sẽ bao gồm việc kiểm tra tính chính xác của các thuật toán (như xử lý va chạm và xoay khối), kiểm tra giao diện người dùng, và kiểm tra hiệu suất của game trên các nền tảng khác nhau. Sau khi phát hiện lỗi, nhóm sẽ tối ưu hóa mã nguồn để cải thiện hiệu suất và đảm bảo trò chơi hoạt động mượt mà.

Đánh giá và hoàn thiện: Cuối cùng, nhóm sẽ đánh giá kết quả thực hiện dựa trên các tiêu chí đã đề ra trong giai đoạn phân tích yêu cầu. Các phản hồi từ người chơi thử

nghiệm sẽ được thu thập và sử dụng để cải thiện giao diện, tính năng và trải nghiệm tổng thể của game. Nhóm cũng sẽ nghiên cứu thêm các cải tiến và tính năng bổ sung cho trò chơi trong tương lai.

4. Ứng dụng nhóm thảo luận, chia sẻ

Hợp tác nhóm: Nhóm sẽ thực hiện các cuộc thảo luận trên google meet và chia sẻ về các tính năng, chiến lược phát triển trò chơi, và các kỹ thuật lập trình để xây dựng và tối ưu hóa trò chơi Tetris. Việc này giúp các thành viên trong nhóm trao đổi kiến thức, học hỏi lẫn nhau và cùng nhau giải quyết các vấn đề kỹ thuật.

Cộng tác trong việc phát triển tính năng: Các thành viên trong nhóm sẽ tham gia vào các phần công việc cụ thể, như lập trình giao diện người dùng (UI), phát triển các tính năng trò chơi như "phá hủy khối", "pause", "bảng xếp hạng", và các hiệu ứng đồ họa.

Đánh giá và cải tiến: Nhóm sẽ tổ chức các buổi chơi thử nghiệm trò chơi để thu thập phản hồi từ người chơi, từ đó cải thiện giao diện và tính năng của trò chơi. Cộng tác với nhóm sẽ giúp đảm bảo rằng tất cả các phần của trò chơi đều được kiểm tra và hoạt động tốt.

Chia sẻ tài nguyên học tập: Các thành viên trong nhóm có thể chia sẻ tài liệu học tập, ví dụ như các khóa học lập trình game, các bài viết về thiết kế trò chơi, hay các công cụ và thư viện hỗ trợ lập trình game.

II. PHẦN NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Giới thiệu tổng quan về python và sơ lược về đồ án

1.1.1. Tổng quan về python

Python là một ngôn ngữ lập trình bậc cao, dễ học và sử dụng, được thiết kế với mục tiêu giúp lập trình viên phát triển các ứng dụng một cách nhanh chóng và hiệu quả. Python nổi bật với cú pháp rõ ràng, dễ đọc và dễ duy trì, làm cho nó trở thành một lựa chọn phổ biến cho các lập trình viên mới bắt đầu cũng như các chuyên gia trong nhiều lĩnh vực khác nhau.

Các đặc điểm nổi bật của Python:

Dễ học và dễ sử dụng: Python có cú pháp đơn giản và dễ hiểu, giúp người học có thể nhanh chóng làm quen với ngôn ngữ này.

Hỗ trợ nhiều thư viện: Python có một kho thư viện phong phú cho các nhu cầu từ xử lý dữ liệu, lập trình web, đến học máy (machine learning).

Được sử dụng rộng rãi: Python là ngôn ngữ phổ biến trong các lĩnh vực như phát triển web, khoa học dữ liệu, trí tuệ nhân tạo (AI), và lập trình hệ thống.

Nhắm đến các ứng dụng đa dạng: Python có thể được sử dụng để phát triển phần mềm, trò chơi, xử lý ảnh, phân tích dữ liệu, và nhiều ứng dụng khác.

1.1.2. Sơ lược về đồ án

Đồ án này là một trò chơi **Tetris** cơ bản được phát triển bằng ngôn ngữ lập trình **Python**, sử dụng thư viện **Pygame** để xây dựng giao diện đồ họa và xử lý sự kiện. Tetris là một trò chơi cổ điển, trong đó người chơi điều khiển các khối hình (tetrominoes) để xếp vào một lưới và tạo thành các hàng đầy, từ đó loại bỏ các hàng đó và ghi điểm. Mục tiêu của trò chơi là hoàn thành các hàng mà không để các khối chồng lên nhau đến đỉnh màn hình.

Mục tiêu của đồ án

Đồ án Tetris có mục tiêu chính là xây dựng một trò chơi Tetris đầy đủ với các tính năng cơ bản và một số tính năng bổ sung, như bảng xếp hạng và hiệu ứng đồ họa. Cụ thể, đồ án sẽ bao gồm các yêu cầu sau:

1. **Giao diện trò chơi:** Màn hình trò chơi chính sẽ bao gồm lưới Tetris, các khối hình Tetris, điểm số, và các nút điều khiển như tạm dừng, thoát trò chơi.
2. **Điều khiển khối:** Người chơi có thể điều khiển các khối bằng các phím mũi tên (trái, phải, lên, xuống) và xoay khối bằng phím lên.
3. **Xử lý va chạm:** Các khối sẽ rơi xuống và khi chạm vào đáy màn hình hoặc chạm vào khối khác, chúng sẽ được "khóa" lại vào lưới.
4. **Xóa hàng:** Khi một hàng đầy, nó sẽ bị xóa và các hàng phía trên sẽ được dịch xuống, giúp người chơi tiếp tục.

5. **Tính điểm:** Người chơi nhận điểm mỗi khi xóa hàng, và điểm sẽ được cộng thêm khi xóa nhiều hàng cùng lúc.
6. **Bảng xếp hạng:** Lưu điểm và hiển thị bảng xếp hạng của người chơi sau mỗi trận đấu.
7. **Hiệu ứng đặc biệt:** Khi người chơi đạt được một thành tích nhất định (ví dụ: lọt vào top 5 hoặc top 1), sẽ có hiệu ứng pháo hoa để chúc mừng.

Các tính năng và chức năng chính trong đồ án

1. Khối Tetris (Tetrominoes):

- Các khối hình Tetris là những hình vuông gồm 4 ô vuông nhỏ. Các khối này có thể xoay và di chuyển trong lưới.
- Các khối được chọn ngẫu nhiên từ một tập hợp 7 loại hình khác nhau, bao gồm S, Z, I, O, J, L, và T.

2. Điều khiển:

- Người chơi có thể sử dụng các phím mũi tên để điều khiển các khối:
 - **Mũi tên trái:** Di chuyển khối sang trái.
 - **Mũi tên phải:** Di chuyển khối sang phải.
 - **Mũi tên xuống:** Làm khối rơi nhanh xuống dưới.
 - **Mũi tên lên:** Xoay khối.
 - **Phím Space:** Thả khối ngay lập tức xuống đáy nếu vị trí hợp lệ.

3. Tính điểm và xếp hạng:

- Người chơi nhận điểm khi xóa hàng.
- Số điểm sẽ được hiển thị trên màn hình và lưu lại vào tệp Rank.txt sau khi trò chơi kết thúc.
- Nếu người chơi lọt vào top 5, hiệu ứng pháo hoa sẽ được hiển thị để chúc mừng thành tích.

4. Tạm dừng và thoát trò chơi:

- Trò chơi có thể tạm dừng và tiếp tục lại thông qua một nút "Pause" ở góc trên bên phải.
- Người chơi có thể thoát trò chơi bất kỳ lúc nào bằng nút "Exit".

5. Bảng xếp hạng:

- Sau khi trò chơi kết thúc, điểm số và vị trí xếp hạng của người chơi sẽ được lưu vào tệp và hiển thị trong bảng xếp hạng.
- Bảng xếp hạng sẽ hiển thị tên và điểm của người chơi theo thứ tự từ cao xuống thấp.

6. Hiệu ứng đồ họa:

- Khi người chơi đạt được thành tích đặc biệt (như lọt vào top 5), hiệu ứng pháo hoa sẽ được tạo ra trên màn hình để chúc mừng người chơi.
- Các khối có thể có màu sắc khác nhau và có thể sử dụng hiệu ứng nhấp nháy khi phá hủy các khối.

Cấu trúc đồ án

- **Lưới trò chơi:** Lưới trò chơi sẽ được chia thành 20 hàng và 10 cột, mỗi ô có kích thước 30x30 pixel. Khối sẽ rơi xuống lưới từ trên xuống và phải được xếp vào đúng vị trí trống.
- **Các đối tượng trong trò chơi:**
 - **Khối (Piece):** Đại diện cho các khối Tetris, mỗi khối có thuộc tính vị trí (x, y), hình dạng (dạng ma trận 2D), và màu sắc.
 - **Lưới (grid):** Lưới Tetris là một ma trận 2D, mỗi ô có thể chứa một màu sắc nếu có khối nằm trong đó, hoặc màu đen nếu ô đó trống.
 - **Vị trí đã khóa (locked_positions):** Lưu các vị trí đã bị chiếm bởi các khối, để kiểm tra xem có thể đặt khối mới hay không.

1.2. Giới thiệu và cách sử dụng của các thư viện sử dụng trong đồ án

1.2.1. Thư viện *pygame*

Tính năng:

- **Xử lý đồ họa:** Pygame cho phép bạn tạo và quản lý cửa sổ đồ họa, vẽ các đối tượng như hình chữ nhật, đường thẳng, hình tròn, và hình ảnh.
- **Quản lý sự kiện:** Lắng nghe và xử lý các sự kiện người dùng như nhấn phím, di chuyển chuột, click chuột, giúp bạn điều khiển trò chơi.
- **Quản lý âm thanh:** Pygame hỗ trợ việc phát nhạc nền và âm thanh hiệu ứng trong trò chơi, giúp tạo thêm tính tương tác và thú vị cho người chơi.
- **Hiển thị văn bản:** Pygame cung cấp chức năng để vẽ các văn bản lên màn hình, giúp hiển thị điểm số, tên người chơi, và thông báo khác.
- **Xử lý hoạt ảnh và hiệu ứng:** Pygame hỗ trợ việc xử lý các hiệu ứng hình ảnh và hoạt ảnh như nhấp nháy, chuyển động, và các hiệu ứng đặc biệt (pháo hoa, tạm dừng, v.v).

Ứng dụng trong đồ án:

- Pygame được sử dụng để xây dựng **giao diện người dùng** cho trò chơi Tetris, bao gồm **lưới trò chơi**, **khối Tetris**, và các nút điều khiển như **Pause**, **Exit**.
- Pygame cũng giúp quản lý việc **vẽ các hình ảnh**, **hiển thị điểm số**, và **xử lý âm thanh nền** (nhạc nền và âm thanh hiệu ứng khi nhấn nút hoặc hoàn thành nhiệm vụ).
- Thư viện này hỗ trợ việc **hiển thị các hiệu ứng đồ họa** (như nhấp nháy các ô khi phá hủy khối hoặc hiệu ứng pháo hoa khi người chơi đạt thành tích cao).

1.2.2. Thư viện *Random*

Tính năng:

- **Sinh số ngẫu nhiên:** Thư viện này cung cấp các hàm sinh số ngẫu nhiên, cho phép bạn lựa chọn ngẫu nhiên các giá trị từ một tập hợp hoặc tạo số ngẫu nhiên trong một khoảng nhất định.

- **Chọn ngẫu nhiên:** `random.choice()` cho phép bạn chọn ngẫu nhiên một phần tử từ một danh sách hoặc chuỗi.

Ứng dụng trong đồ án:

- Trong trò chơi Tetris, `random` được sử dụng để **chọn khối Tetris ngẫu nhiên** mỗi khi người chơi bắt đầu một vòng chơi mới. Mỗi khối có thể là một trong bảy hình dạng khác nhau, và thư viện `random` giúp việc chọn lựa này trở nên ngẫu nhiên.
- Nó cũng có thể được dùng để **xác định số lượng khối bị phá hủy ngẫu nhiên** khi người chơi sử dụng tính năng phá hủy.

1.2.3. Thư viện *Math*

Tính năng:

- **Các phép toán toán học cơ bản và nâng cao:** Thư viện `math` cung cấp nhiều hàm toán học như `sin`, `cos`, `sqrt`, `pi`, v.v.
- **Hàm xử lý số học phức tạp:** Các hàm giúp tính toán góc, khoảng cách, và các phép toán khác trong các bài toán liên quan đến đồ họa và vật lý.

Ứng dụng trong đồ án:

- **Xử lý góc quay của khối Tetris:** Các khối Tetris có thể xoay trong không gian. Hàm `math` giúp tính toán vị trí của các ô khi khối xoay, đặc biệt khi tính toán góc xoay cho các khối dạng chữ "T" hoặc "L".
- **Hiệu ứng pháo hoa:** Trong phần hiệu ứng pháo hoa, thư viện `math` giúp tính toán **vận tốc và hướng di chuyển** của các hạt pháo hoa, sử dụng các phép toán như `sin` và `cos` để xác định hướng di chuyển ngẫu nhiên của các hạt.

1.2.4. Thư viện *Pathlib*

Tính năng:

- **Quản lý đường dẫn tệp tin:** `pathlib` giúp tạo, quản lý và thao tác với các đường dẫn tệp và thư mục.

- **Kiểm tra sự tồn tại của tệp:** Cung cấp các phương thức như Path.exists() để kiểm tra xem tệp hay thư mục có tồn tại hay không.
- **Tạo thư mục, tệp:** pathlib hỗ trợ tạo các thư mục hoặc tệp mới khi cần thiết.

Ứng dụng trong đồ án:

- **Lưu và đọc dữ liệu bảng xếp hạng:** Thư viện pathlib được sử dụng để quản lý đường dẫn tệp đến tệp Rank.txt nơi lưu trữ điểm số và xếp hạng của người chơi.
- **Quản lý ảnh nền và âm thanh:** pathlib cũng được sử dụng để xử lý đường dẫn đến các tệp hình ảnh (nền, các khối) và âm thanh (nhạc nền, âm thanh nút bấm) trong trò chơi.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1. Đặc tả yêu cầu

2.1.1. Yêu cầu chức năng

Yêu cầu chức năng mô tả các tính năng mà hệ thống phải thực hiện để đáp ứng nhu cầu của người dùng và yêu cầu kỹ thuật. Dưới đây là các yêu cầu chức năng cơ bản của trò chơi Tetris:

- Quản lý các khối Tetris:

- Hệ thống phải tạo ra các khối Tetris ngẫu nhiên (S, Z, I, O, J, L, T) và di chuyển chúng xuống lưới.
- Mỗi khối phải có các chức năng xoay, di chuyển trái/phải và rơi xuống tự động.

- Kiểm tra va chạm và hợp lệ:

- Kiểm tra xem khối Tetris có hợp lệ khi di chuyển hay xoay không (kiểm tra va chạm với các khối khác và viền của lưới).
- Nếu khối chạm đáy hoặc va chạm với khối khác, hệ thống sẽ khóa khối và tạo ra một khối mới.

- Xử lý tính năng tạm dừng:

- Người chơi có thể tạm dừng trò chơi bất cứ lúc nào (nút "Pause").
- Khi trò chơi tạm dừng, hệ thống phải ngừng mọi hoạt động của khối và tạm dừng âm thanh.

- Xử lý tính năng phá hủy khối:

- Người chơi có thể chọn các khối trên lưới để phá hủy thông qua tính năng "Destroy" khi nhấn chuột.
- Hệ thống phải xóa các khối được chọn và cập nhật lại lưới.

- Cập nhật điểm số:

- Điểm số của người chơi phải được tính dựa trên số lượng hàng được xóa.
- Hệ thống phải lưu điểm số và xếp hạng người chơi vào một file (Rank.txt).

- Bảng xếp hạng và lưu trữ điểm:

- Lưu điểm số của người chơi vào bảng xếp hạng và hiển thị bảng xếp hạng sau khi trò chơi kết thúc.
- Cung cấp thông tin về xếp hạng người chơi và hiển thị pháo hoa khi người chơi đạt vị trí Top 5.

- Giao diện người dùng:

- Giao diện trò chơi phải bao gồm các thành phần cơ bản như:
 - Lưới trò chơi (grid).
 - Các nút như "Pause", "Exit".
 - Thông báo điểm số, số lần phá hủy còn lại.
 - Hướng dẫn nhập tên người chơi.

- Hiệu ứng đồ họa:

- Cung cấp các hiệu ứng khi người chơi đạt được một thành tích nhất định, ví dụ như hiệu ứng pháo hoa khi vào Top 5 hoặc Top 1.

- Cài đặt độ khó:

- Cung cấp cho người chơi ba cấp độ độ khó: Easy, Normal, Hard, với tốc độ rơi khối thay đổi theo cấp độ.

- Chế độ chơi và thoát trò chơi:

- Người chơi có thể bắt đầu trò chơi mới, tạm dừng hoặc thoát trò chơi từ menu chính.

2.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng mô tả các yếu tố không liên quan trực tiếp đến tính năng nhưng lại rất quan trọng trong việc đảm bảo hệ thống hoạt động hiệu quả. Các yêu cầu phi chức năng của hệ thống bao gồm:

- Hiệu suất:

- Trò chơi phải chạy mượt mà trên các máy tính có cấu hình phần cứng thấp, đảm bảo ít nhất 30 FPS (frames per second).
- Các hiệu ứng đồ họa như pháo hoa không được làm giảm hiệu suất của trò chơi.

- Khả năng mở rộng:

- Hệ thống phải dễ dàng mở rộng để có thể thêm các tính năng mới trong tương lai, chẳng hạn như thêm các chế độ chơi mới, hoặc tối ưu hóa thuật toán cho AI.

- Tính ổn định:

- Trò chơi phải không bị crash trong các tình huống thường gặp, chẳng hạn như khi di chuyển khối, xoay khối, hoặc khi người chơi thực hiện các thao tác phá hủy khối.

- Tính tương thích:

- Trò chơi phải tương thích với các hệ điều hành phổ biến như Windows, macOS, và Linux.
- Hệ thống phải hỗ trợ đa dạng các loại màn hình với độ phân giải từ 1024x768 trở lên.

- Giao diện người dùng (UI):

- Giao diện người chơi phải dễ sử dụng, với các nút bấm rõ ràng và dễ hiểu.
- Màu sắc và thiết kế của giao diện phải phù hợp với chủ đề của trò chơi (Tetris cổ điển).

- Bảo mật dữ liệu:

- Dữ liệu điểm số và bảng xếp hạng phải được bảo mật và không dễ bị sửa đổi.

2.2. Đặc tả luồng chức năng

2.2.1. Luồng chính

- Khởi tạo trò chơi:

- Hệ thống khởi tạo môi trường trò chơi, bao gồm việc tải ảnh nền, thiết lập lưới trò chơi, và thiết lập các giá trị mặc định như tốc độ rơi của khối, số lần phá hủy tối đa, điểm số, v.v.

- Bắt đầu trò chơi:

- Người chơi bắt đầu một trò chơi mới từ menu chính.

- Hệ thống tạo một khối Tetris mới và di chuyển nó xuống lưới.
- Trò chơi tiếp tục cho đến khi khối chạm đáy hoặc có một va chạm.

- Xử lý sự kiện người chơi:

- Hệ thống nhận và xử lý các sự kiện như nhấn phím (di chuyển, xoay khối, tạm dừng) và nhấn chuột (phá hủy khối).

- Quản lý độ khó và tốc độ rơi:

- Người chơi có thể thay đổi độ khó trong phần cài đặt.
- Hệ thống điều chỉnh tốc độ rơi của các khối tùy thuộc vào mức độ khó được chọn.

- Cập nhật và hiển thị thông tin:

- Hệ thống liên tục cập nhật trạng thái của trò chơi, hiển thị điểm số, số lần phá hủy còn lại, và các hiệu ứng đồ họa nếu có.
- Trò chơi cập nhật bảng xếp hạng và hiển thị pháo hoa khi người chơi vào Top 5 hoặc Top 1.

- Kết thúc trò chơi:

- Khi người chơi thua hoặc tự ý thoát trò chơi, hệ thống sẽ lưu điểm và xếp hạng vào bảng xếp hạng, sau đó chuyển đến màn hình kết quả.

2.2.2. Luồng phụ

- Cài đặt độ khó:

- Người chơi có thể thay đổi độ khó trong menu cài đặt.
- Khi thay đổi độ khó, hệ thống sẽ điều chỉnh tốc độ rơi của các khối và lưu lại cài đặt.

- Tính năng "Phá hủy khối":

- Người chơi có thể sử dụng chuột để phá hủy các khối đã khóa trong lưới.
- Hệ thống sẽ xóa các khối được chọn và cập nhật lại lưới, sau đó giảm số lần phá hủy còn lại.

- Tạm dừng trò chơi:

- Người chơi có thể tạm dừng trò chơi bất cứ lúc nào.
- Khi tạm dừng, hệ thống ngừng mọi hoạt động và giữ trạng thái hiện tại của trò chơi.

- Hiển thị bảng xếp hạng:

- Sau khi trò chơi kết thúc, bảng xếp hạng sẽ được hiển thị cho người chơi.
- Nếu người chơi nằm trong Top 5, hệ thống sẽ hiển thị các hiệu ứng pháo hoa.

CHƯƠNG 3: THIẾT KẾ VÀ PHÁT TRIỂN HỆ THỐNG

3.1. Thiết kế giao diện

Giao diện người dùng (UI - User Interface) là một phần quan trọng trong quá trình phát triển game và có nhiều lý do để lý giải vì sao giao diện cần thiết trong việc viết mã cho game.

Giao diện giúp người chơi tương tác với trò chơi một cách trực quan và dễ dàng. UI cung cấp các hướng dẫn, thông tin và tín hiệu trực tiếp cho người chơi (ví dụ: cách chơi, điểm số, thời gian, v.v.).

Người chơi có thể xem điểm số, tiến độ, và các chỉ số quan trọng khác trong game. Hiển thị các thông báo quan trọng như "You Lost",... hoặc bất kỳ sự kiện nào xảy ra trong game.

Các nút bấm và các yếu tố tương tác khác giúp người chơi điều khiển và thao tác trong game. Cho phép người chơi điều hướng qua các tùy chọn, thiết lập cấu hình, và bắt đầu hoặc dừng trò chơi.

Một giao diện hấp dẫn có thể làm cho trò chơi trông chuyên nghiệp và cuốn hút hơn. Giao diện góp phần xây dựng hình ảnh và phong cách riêng của trò chơi.

Người chơi nhanh chóng nắm bắt được thông tin cần thiết, từ đó cải thiện hiệu suất chơi. UI cung cấp phản hồi tức thời về hành động của người chơi, giúp họ điều chỉnh chiến lược và cách chơi.

Giao diện giúp tích hợp âm thanh hiệu ứng và nhạc nền, tăng thêm trải nghiệm thực tế cho người chơi. Các hiệu ứng chuyển động, ánh sáng và đồ họa giúp làm cho trò chơi sống động hơn.

3.1.1. Menu chính



Hình 1: Menu chính

Giao diện này là màn hình chính của game TETRIS. Nền của giao diện có các hình khối đầy màu sắc, gợi nhớ đến các mảnh ghép Tetris, một số khối còn có các khuôn mặt. Tiêu đề "TETRIS" được hiển thị nổi bật ở phía trên cùng với phong chữ đa màu sắc sống động. Bên dưới tiêu đề, có bốn tùy chọn trong menu: "Play", "Level", "Rankings" và "Quit".

Giao diện này có tính thẩm mỹ hấp dẫn và đầy màu sắc, tạo sự thu hút cho người dùng. Nền của hình ảnh còn hiển thị một môi trường mã hóa, cho thấy đây có thể là phiên bản tùy chỉnh hoặc phiên bản phát triển của trò chơi.

- Nút "Play"

- **Chức năng:** Bắt đầu trò chơi.
- **Vị trí:** Thường được đặt ở vị trí trung tâm, dễ thấy.
- **Giao diện:** Nút có chữ "Play" màu đen trên nền trắng.
- **Phản hồi:** Khi người dùng nhấp vào, trò chơi sẽ bắt đầu và chuyển sang màn hình chơi game.

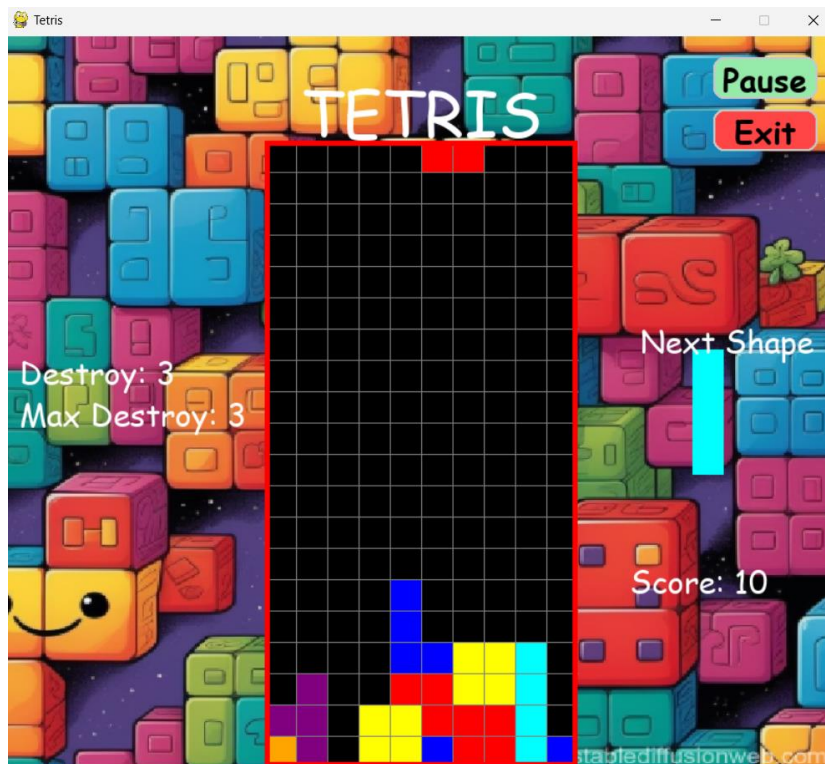
- **Nút "Level"**
 - **Chức năng:** Mở cài đặt trò chơi.
 - **Vị trí:** Thường nằm dưới hoặc bên cạnh nút "Play".
 - **Giao diện:** Nút có chữ "Level" màu đen trên nền trắng.
 - **Phản hồi:** Khi người dùng nhấp vào, sẽ hiển thị các tùy chọn cài đặt như độ khó, âm thanh, và tùy chỉnh giao diện.
- **Nút "Rankings"**
 - **Chức năng:** Hiển thị bảng xếp hạng người chơi.
 - **Vị trí:** Thường nằm bên dưới hoặc cạnh nút "Settings".
 - **Giao diện:** Nút có chữ "Rankings" màu đen trên nền trắng.
 - **Phản hồi:** Khi người dùng nhấp vào, sẽ hiển thị danh sách các điểm số cao và tên người chơi.
- **Nút "Quit"**
 - **Chức năng:** Thoát khỏi trò chơi.
 - **Vị trí:** Thường nằm dưới cùng của danh sách nút.
 - **Giao diện:** Nút có chữ "Quit" màu đen trên nền trắng.
 - **Phản hồi:** Khi người dùng nhấp vào, trò chơi sẽ thoát ra và trở về màn hình nền hoặc đóng ứng dụng.
- **Hình ảnh và Giao diện**
 - Các nút đều có hình chữ nhật với màu nền trắng và chữ đen, tạo sự tương phản rõ rệt và dễ nhìn.
 - Khi người dùng di chuột qua các nút, màu sắc hoặc viền của nút có thể thay đổi để chỉ ra rằng nút đó có thể tương tác được.
 - Các nút đều được sắp xếp một cách hài hòa, dễ tìm kiếm và dễ sử dụng, góp phần vào trải nghiệm người dùng mượt mà và thân thiện.

3.1.2. Giao diện trò chơi



Hình 2: Nhập tên

Khi người chơi vào trò chơi hệ thống game đề xuất tính năng nhập tên người chơi để lưu lại hạng của người chơi.



Hình 3: Vào chơi

Vào trò chơi các khối Tetris được xếp ở dưới cùng của lưới chơi chính

- **Tiêu đề:** Phía trên cùng của màn hình có tiêu đề "TETRIS" được viết bằng phông chữ lớn và nổi bật.
- **Khu vực chơi chính:** Khu vực chơi chính là một lưới đen, nơi các khối Tetris được xếp chồng lên nhau. Hiện tại, các khối đã xếp chạm đến khoảng giữa của lưới.

- **Khối tiếp theo:** Phía bên phải của lưới chính, có một khu vực hiển thị hình dạng của khối Tetris tiếp theo. Khối tiếp theo là một đường thẳng màu cyan.
- **Điểm số và Thông tin:** Góc trên cùng bên phải có hiển thị "Score: 0", chỉ ra rằng điểm số hiện tại là 0. Bên trái màn hình có các thông tin "Destroy: 3" và "Max Destroy: 3", cho biết số lần phá hủy còn lại và số lần phá hủy tối đa.
- **Các nút điều khiển:** Góc trên cùng bên phải có hai nút "Pause" và "Exit", cho phép người chơi tạm dừng hoặc thoát khỏi trò chơi.



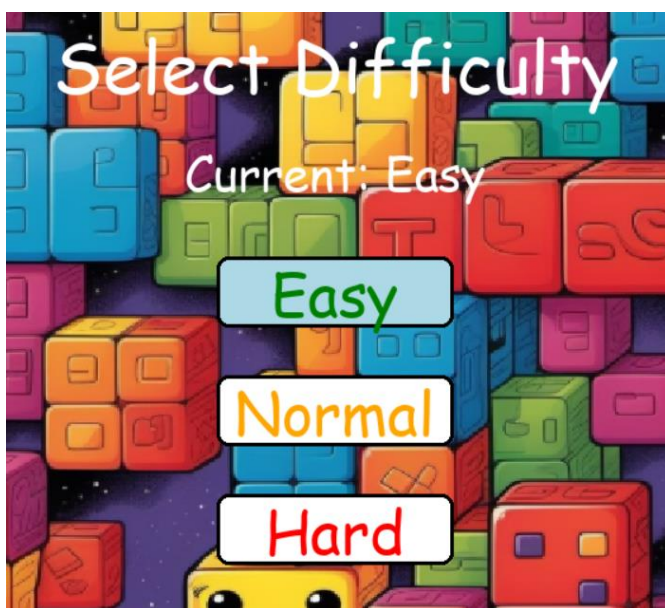
Hình 4: Top rankings

Sau khi kết thúc trò chơi, giao diện sẽ hiển thị bảng xếp hạng "Top Rankings" với tên và điểm số của người chơi.

Danh sách bao gồm tên và điểm số của người chơi được sắp xếp theo thứ tự từ cao đến thấp.

Bảng xếp hạng giúp người chơi theo dõi thành tích của mình và so sánh với những người chơi khác, từ đó tăng tính cạnh tranh và động lực để cải thiện kỹ năng chơi game.

3.1.3. Menu settings



Hình 5. Select Difficulty

Trong phần Menu Settings trò chơi Tetris, nơi người chơi có thể chọn mức độ khó. Nền của giao diện gồm các hình khối đầy màu sắc, gợi nhớ đến các mảnh ghép Tetris.

- **Tiêu đề:** Phía trên cùng của menu có tiêu đề "Select Difficulty" (Chọn mức độ khó) được viết bằng phông chữ lớn và rõ ràng.
- **Mức độ khó hiện tại:** Dưới tiêu đề có dòng chữ "Current: Easy" (Hiện tại: Dễ), cho biết mức độ khó hiện tại của trò chơi.
- **Các nút chọn mức độ khó:** Có ba nút để chọn mức độ khó khác nhau:
 1. **Easy (Dễ):**
 - Nút này được tô màu xanh lá cây và hiện đang được chọn.
 2. **Normal (Bình thường):**
 - Nút này được tô màu cam.
 3. **Hard (Khó):**
 - Nút này được tô màu đỏ.

+ Các nút được sắp xếp theo chiều dọc ở trung tâm của hình ảnh.

Giao diện này giúp người chơi dễ dàng thay đổi mức độ khó của trò chơi, tạo nên trải nghiệm chơi game đa dạng và phù hợp với khả năng của từng người chơi.

3.2. Thiết kế chức năng

Mục tiêu của mục này là mô tả chi tiết các chức năng của trò chơi Tetris, bao gồm các tính năng chính và cách thức chúng hoạt động để đảm bảo người chơi có trải nghiệm tốt nhất.

3.2.1. Hàm logic

- Class Piece là nền tảng để định nghĩa và quản lý các khối trong trò chơi Tetris. Các thuộc tính này giúp xác định vị trí, hình dạng, màu sắc và góc quay của khối, cho phép các hàm khác trong trò chơi tương tác với khối một cách hiệu quả và chính xác.

```
175 class Piece(object):
176     rows = 20 # y
177     columns = 10 # x
178
179     def __init__(self, column, row, shape):
180         self.x = column
181         self.y = row
182         self.shape = shape
183         self.color = shape_colors[shapes.index(shape)]
184         self.rotation = 0 # number from 0-3
```

Hình 6. Class piece

- + **rows = 20**: Đặt số hàng của lưới chơi là 20.
- + **columns = 10**: Đặt số cột của lưới chơi là 10.
- + **self.x = column**: Thiết lập tọa độ x (cột) của khối.
- + **self.y = row**: Thiết lập tọa độ y (hàng) của khối.
- + **self.shape = shape**: Thiết lập hình dạng của khối.
- + **self.color = shape_colors[shapes.index(shape)]**: Thiết lập màu sắc của khối dựa trên hình dạng của nó.
- + **self.rotation = 0**: Đặt góc quay của khối là 0 (khởi tạo ban đầu, có thể thay đổi từ 0 đến 3).

```

186 def set_difficulty(level):
187     global difficulty, fall_speed
188     difficulty = level
189     if level == "Easy":
190         fall_speed = 1 # Rơi chậm
191     elif level == "Normal":
192         fall_speed = 0.5 # Rơi trung bình
193     elif level == "Hard":
194         fall_speed = 0.1 # Rơi nhanh

```

Hình 7. Hàm *set_difficulty*

- + **global difficulty, fall_speed**: Sử dụng các biến toàn cục *difficulty* và *fall_speed*
- + **difficulty = level**: Đặt mức độ khó của trò chơi.
- + **if level == "Easy"**: Kiểm tra nếu mức độ khó là "Dễ".
- + **fall_speed = 1**: Đặt tốc độ rơi của khối là 1 (rơi chậm).
- + **elif level == "Normal"**: Kiểm tra nếu mức độ khó là "Bình thường".
- + **fall_speed = 0.5**: Đặt tốc độ rơi của khối là 0.5 (rơi trung bình).
- + **elif level == "Hard"**: Kiểm tra nếu mức độ khó là "Khó".
- + **fall_speed = 0.1**: Đặt tốc độ rơi của khối là 0.1 (rơi nhanh).

```

199 def create_grid(locked_positions={}):
200     grid = [(0, 0, 0) for _ in range(10)] for _ in range(20)]
201     for i in range(len(grid)):
202         for j in range(len(grid[i])):
203             if (j, i) in locked_positions:
204                 c = locked_positions[(j, i)]
205                 grid[i][j] = c
206     return grid

```

Hình 8. Hàm *create_grid*

- + **grid = [(0, 0, 0) for _ in range(10)] for _ in range(20)]**: Tạo lưới 20 hàng và 10 cột với tất cả các ô đều có màu đen.
- + **for i in range(len(grid))**: Duyệt qua từng hàng của lưới.
- + **for j in range(len(grid[i]))**: Duyệt qua từng cột trong hàng.
- + **if (j, i) in locked_positions**: Kiểm tra nếu vị trí (j, i) đã bị khóa.
- + **c = locked_positions[(j, i)]**: Lấy màu của khối tại vị trí bị khóa.
- + **grid[i][j] = c**: Đặt màu của khối vào lưới.

- + **return grid**: Trả về lưới đã được cập nhật.

```

210 ✓ def convert_shape_format(shape):
211     positions = []
212     format = shape.shape[shape.rotation % len(shape.shape)]
213
214 ✓     for i, line in enumerate(format):
215         row = list(line)
216 ✓         for j, column in enumerate(row):
217 ✓             if column == '0':
218                 positions.append((shape.x + j, shape.y + i))
219
220 ✓     for i, pos in enumerate(positions):
221         positions[i] = (pos[0] - 2, pos[1] - 4)
222
223     return positions

```

Hình 9. Hàm `convert_shape_format`

- + **positions = []**: Khởi tạo danh sách để lưu các vị trí của khối.
- + **format = shape.shape[shape.rotation % len(shape.shape)]**: Lấy định dạng của hình dạng khối dựa trên góc quay hiện tại.
- + **for i, line in enumerate(format)**: Duyệt qua từng dòng trong định dạng khối.
- + **row = list(line)**: Chuyển đổi dòng thành danh sách các ký tự.
- + **for j, column in enumerate(row)**: Duyệt qua từng cột trong dòng.
- + **if column == '0'**: Kiểm tra các vị trí có giá trị '0' để xác định các ô trong khối.
- + **positions.append((shape.x + j, shape.y + i))**: Thêm vị trí của các ô vào danh sách positions.
- + **positions[i] = (pos[0] - 2, pos[1] - 4)**: Điều chỉnh vị trí của các ô để phù hợp với lưới chơi.
- + **return positions**: Trả về danh sách các vị trí của khối.

```

226 def valid_space(shape, grid):
227     accepted_positions = [[(j, i) for j in range(10) if grid[i][j] == (0,0,0)] for i in range(20)]
228     accepted_positions = [j for sub in accepted_positions for j in sub]
229     formatted = convert_shape_format(shape)
230
231     for pos in formatted:
232         if pos not in accepted_positions:
233             if pos[1] > -1:
234                 return False

```

Hình 10. Hàm `valid_space`

- + **accepted_positions = [(j, i) for j in range(10) if grid[i][j] == (0,0,0)] for i in range(20)**: Tạo danh sách các vị trí hợp lệ trong lưới chơi.

- + **accepted_positions = [j for sub in accepted_positions for j in sub]:** Chuyển đổi danh sách 2D thành danh sách 1D.
- + **formatted = convert_shape_format(shape):** Chuyển đổi định dạng của khối thành các tọa độ cụ thể.
- + **for pos in formatted:** Duyệt qua từng vị trí trong khối.
- + **if pos not in accepted_positions:** Kiểm tra xem vị trí của khối có nằm trong danh sách các vị trí hợp lệ không.
- + **if pos[1] > -1:** Kiểm tra xem vị trí y của khối có lớn hơn -1 không.
- + **return False:** Trả về False nếu vị trí không hợp lệ.
- + **return True:** Trả về True nếu vị trí hợp lệ.

```

239 def check_lost(locked):
240     for pos in locked:
241         x, y = pos
242         if y < 1: # Kiểm tra nếu có khối nào nằm ở hàng đầu tiên
243             return True
244     return False

```

Hình 11. Hàm *check_lost*

- + **for pos in locked:** Duyệt qua từng vị trí đã khóa.
- + **x, y = pos:** Lấy tọa độ x và y của vị trí.
- + **if y < 1:** Kiểm tra nếu có khối nào nằm ở hàng đầu tiên.
- + **return True:** Trả về True nếu trò chơi kết thúc.
- + **return False:** Trả về False nếu trò chơi chưa kết thúc.

```

249 def get_shape():
250     global shapes, shape_colors
251     # Khởi tạo khối ở giữa hàng trên cùng (y = 0)
252     return Piece(5, 0, random.choice(shapes))
253

```

Hình 12. Hàm *get_shape*

- + **global shapes, shape_colors:** Sử dụng các biến toàn cục shapes và shape_colors.
- + **return Piece(5, 0, random.choice(shapes)):** Tạo một khối mới với hình dạng ngẫu nhiên và tọa độ ban đầu (cột 5, hàng 0).

```

298 def clear_rows(grid, locked):
299     # need to see if row is clear the shift every other row above down one
300     global score
301
302     inc = 0
303     for i in range(len(grid)-1,-1,-1):
304         row = grid[i]
305         if (0, 0, 0) not in row:
306             inc += 1
307             # add positions to remove from locked
308             ind = i
309             for j in range(len(row)):
310                 try:
311                     del locked[(j, i)]
312                 except:
313                     continue
314     if inc > 0:
315         for key in sorted(list(locked), key=lambda x: x[1])[::-1]:
316             x, y = key
317             if y < ind:
318                 newKey = (x, y + inc)
319                 locked[newKey] = locked.pop(key)
320
321     score += inc * 10

```

Hình 13. Hàm `clear_rows`

- + `global score`: Sử dụng biến toàn cục `score`.
- + `for i in range(len(grid)-1,-1,-1)`: Duyệt qua từng hàng của lưới từ dưới lên trên.
- + `if (0, 0, 0) not in row`: Kiểm tra xem hàng có đầy đủ không (không chứa màu đen).
- + `del locked[(j, i)]`: Xóa vị trí đã khóa trong hàng.
- + `score += inc * 10`: Cập nhật điểm số dựa trên số lượng hàng đã xóa.

```

348 def toggle_pause():
349     global paused
350     paused = not paused

```

Hình 14. Hàm `toggle_pause`

- + `global paused`: Sử dụng biến toàn cục `paused`.
- + `paused = not paused`: Chuyển đổi trạng thái tạm dừng.

```

411 def button_hover(x, y, width, height, mouse_pos):
412     mouse_x, mouse_y = mouse_pos
413     return x <= mouse_x <= x + width and y <= mouse_y <= y + height
414

```

Hình 15. Hàm `button_hover`

- + Tách tuple `mouse_pos` thành hai biến riêng biệt: `mouse_x` và `mouse_y`, tương ứng với tọa độ x và y của con trỏ chuột.
- + Điều kiện `x <= mouse_x <= x + width` kiểm tra xem tọa độ x của chuột có nằm trong phạm vi chiều ngang của nút bấm (từ x đến x + width).
- + Điều kiện `y <= mouse_y <= y + height` kiểm tra xem tọa độ y của chuột có nằm trong phạm vi chiều dọc của nút bấm (từ y đến y + height).

```

482 def destroy_blocks(locked_positions):
483     # Xóa ngẫu nhiên 3-5 khối từ các vị trí đã khóa
484     num_blocks_to_destroy = random.randint(3, 5) # Số lượng khối phá hủy ngẫu nhiên
485     positions_to_destroy = random.sample(list(locked_positions.keys()), min(num_blocks_to_destroy, len(locked_positions)))
486
487     # Xóa các khối khỏi vị trí đã khóa
488     for pos in positions_to_destroy:
489         del locked_positions[pos]
490
491     # Cập nhật lại Lưới
492     global grid
493     grid = create_grid(locked_positions)

```

Hình 16. Hàm `destroy_blocks`

- + **`random.randint(3, 5)`**: Tạo một số ngẫu nhiên từ 3 đến 5 để quyết định số lượng khối sẽ bị phá hủy.
- + **`random.sample(list(locked_positions.keys()), min(num_blocks_to_destroy, len(locked_positions)))`**: Chọn ngẫu nhiên các vị trí để phá hủy từ danh sách các vị trí đã khóa.
- + **`for pos in positions_to_destroy: del locked_positions[pos]`**: Xóa các khối khỏi vị trí đã khóa.
- + **`global grid`**: Khai báo biến toàn cục `grid`.
- + **`grid = create_grid(locked_positions)`**: Cập nhật lại lưới dựa trên các vị trí đã khóa còn lại.

```

495 def destroy_blocks_with_effect_at_mouse(surface, locked_positions, mouse_pos):
496     global grid, destroy_count # Cập nhật biến toàn cục
497
498     # Kiểm tra số lần phá hủy còn lại
499     if destroy_count >= max_destroy:
500         print("Bạn đã sử dụng hết số lần phá hủy!")
501         return
502
503     # Xác định vị trí chuột trong lưới
504     mouse_x, mouse_y = mouse_pos
505     grid_x = (mouse_x - top_left_x) // block_size
506     grid_y = (mouse_y - top_left_y) // block_size
507
508     # Kiểm tra vị trí chuột có nằm trong phạm vi lưới không
509     if grid_x < 0 or grid_x >= 10 or grid_y < 0 or grid_y >= 20:
510         print("Vị trí chuột ngoài phạm vi lưới!")
511         return
512
513     # Tạo vùng phá hủy 3x3 xung quanh vị trí chuột
514     positions_to_destroy = []
515     for dx in range(-1, 2): # -1, 0, 1
516         for dy in range(-1, 2): # -1, 0, 1
517             x, y = grid_x + dx, grid_y + dy
518             if (x, y) in locked_positions: # Chỉ thêm vị trí hợp lệ
519                 positions_to_destroy.append((x, y))
520

```

Hình 17. Hàm `destroy_blocks_with_effectmouse`

- + **global grid, destroy_count**: Cập nhật biến toàn cục.
- + **if destroy_count >= max_destroy**: Kiểm tra số lần phá hủy còn lại.
- + **mouse_x, mouse_y = mouse_pos; grid_x = (mouse_x - top_left_x) // block_size; grid_y = (mouse_y - top_left_y) // block_size**: Xác định vị trí chuột trong lưới.
- + **if grid_x < 0 or grid_x >= 10 or grid_y < 0 or grid_y >= 20**: Kiểm tra vị trí chuột có nằm trong phạm vi lưới không.
- + **positions_to_destroy = []; for dx in range(-1, 2); for dy in range(-1, 2)**: Tạo vùng phá hủy 3x3 xung quanh vị trí chuột.
- + **if (x, y) in locked_positions**: Chỉ thêm vị trí hợp lệ vào danh sách phá hủy.

```

522     for pos in positions_to_destroy:
523         x, y = pos
524         color = locked_positions[pos]
525
526         # Hiệu ứng nhấp nháy
527         for _ in range(3): # Lặp hiệu ứng nhấp nháy 3 lần
528             pygame.draw.rect(surface, color,
529                             (top_left_x + x * block_size, top_left_y + y * block_size, block_size, block_size))
530             pygame.display.update()
531             pygame.time.delay(100)
532             pygame.draw.rect(surface, (0, 0, 0),
533                             (top_left_x + x * block_size, top_left_y + y * block_size, block_size, block_size))
534             pygame.display.update()
535             pygame.time.delay(100)
536
537         # Xóa vị trí khỏi danh sách bị khóa
538         del locked_positions[pos]
539
540     # Cập nhật lại Lưới
541     grid = create_grid(locked_positions)
542
543     # Tăng số lần phá hủy đã sử dụng
544     destroy_count += 1
545     print(f"Số lần phá hủy đã sử dụng: {destroy_count}/{max_destroy}")

```

Hình 17.1. Hàm `destroy_blocks_with_effectmouse`

- + **for pos in positions_to_destroy; for _ in range(3):** Hiệu ứng nhấp nháy 3 lần và xóa vị trí khỏi danh sách bị khóa.
- + **grid = create_grid(locked_positions):** Cập nhật lại lưới.
- + **destroy_count += 1:** Tăng số lần phá hủy đã sử dụng.

```

562 def fireworks_effect_pro(surface, num_fireworks=10, duration=2000):
563     start_time = pygame.time.get_ticks()
564     while pygame.time.get_ticks() - start_time < duration:
565         surface.fill((0, 0, 0)) # Làm mới màn hình
566         for _ in range(num_fireworks):
567             x = random.randint(0, s_width)
568             y = random.randint(0, s_height // 2)
569             color = random.choice([(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0)])
570             radius = random.randint(5, 15)
571             pygame.draw.circle(surface, color, (x, y), radius)
572         pygame.display.update()
573         pygame.time.delay(100) # Tạo độ mượt cho hiệu ứng

```

Hình 18. Hàm `fireworks_effect_pro`

- + **start_time = pygame.time.get_ticks():** Lấy thời gian bắt đầu của hiệu ứng.
- + **while pygame.time.get_ticks() - start_time < duration:** Vòng lặp chạy trong khoảng thời gian đã chỉ định (2000 milliseconds).
- + **surface.fill((0, 0, 0)):** Làm mới màn hình (xóa sạch màn hình với màu đen).
- + **for _ in range(num_fireworks):** Lặp lại số lần pháo hoa được tạo ra (10 lần theo mặc định).
- + **x = random.randint(0, s_width); y = random.randint(0, s_height // 2):** Tạo ngẫu nhiên vị trí x và y cho pháo hoa trong nửa trên của màn hình.

- + **color = random.choice([(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0)])**: Chọn màu ngẫu nhiên cho pháo hoa từ một tập hợp màu sắc.
- + **radius = random.randint(5, 15)**: Tạo ngẫu nhiên bán kính của pháo hoa.
- + **pygame.draw.circle(surface, color, (x, y), radius)**: Vẽ một hình tròn (pháo hoa) trên màn hình với màu và bán kính đã chọn.
- + **pygame.display.update()**: Cập nhật màn hình để hiển thị pháo hoa.
- + **pygame.time.delay(100)**: Tạo độ trễ ngắn (100 milliseconds) để hiệu ứng trông mượt mà hơn.

```

575 def fireworks_effect(surface, num_fireworks=10, duration=3000):
576     particles = [] # Danh sách các hạt pháo hoa
577     start_time = pygame.time.get_ticks()
578
579     while pygame.time.get_ticks() - start_time < duration:
580         # Tạo pháo hoa mới
581         if len(particles) < num_fireworks:
582             x = random.randint(100, s_width - 100)
583             y = random.randint(100, s_height // 2)
584             color = random.choice([(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0)])
585             for _ in range(50): # Số lượng hạt mỗi pháo hoa
586                 angle = random.uniform(0, 2 * math.pi) # Góc ngẫu nhiên
587                 speed = random.uniform(2, 5) # Tốc độ ngẫu nhiên
588                 particles.append({
589                     "x": x,
590                     "y": y,
591                     "vx": math.cos(angle) * speed,
592                     "vy": math.sin(angle) * speed,
593                     "radius": random.randint(2, 4), # Kích thước hạt
594                     "color": color,
595                     "lifetime": random.randint(30, 60) # Tuổi thọ hạt
596                 })

```

Hình 19. Hàm *fireworks_effect*

- + **particles = []**: Tạo danh sách rỗng để chứa các hạt pháo hoa.
- + **start_time = pygame.time.get_ticks()**: Lấy thời gian bắt đầu của hiệu ứng.
- + **while pygame.time.get_ticks() - start_time < duration**: Vòng lặp chạy trong khoảng thời gian đã chỉ định (3000 milliseconds).
- + **if len(particles) < num_fireworks**: Tạo pháo hoa mới nếu số lượng hạt hiện tại ít hơn số lượng pháo hoa mong muốn.
- + **random.randint(100, s_width - 100); random.randint(100, s_height // 2)**: Tạo ngẫu nhiên vị trí x và y cho pháo hoa trong khoảng xác định.
- + **random.choice([(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0)])**: Chọn màu ngẫu nhiên cho pháo hoa từ một tập hợp màu sắc.

- + **random.uniform(0, 2 * math.pi); random.uniform(2, 5):** Tạo góc và tốc độ ngẫu nhiên cho hạt pháo hoa.
- + **particles.append(...):** Thêm hạt pháo hoa vào danh sách particles.

```

598     # Vẽ các hạt pháo hoa
599     surface.fill((0, 0, 0)) # Làm sạch màn hình
600     for particle in particles[:]:
601         particle["x"] += particle["vx"]
602         particle["y"] += particle["vy"]
603         particle["lifetime"] -= 1
604         pygame.draw.circle(surface, particle["color"], (int(particle["x"]), int(particle["y"])), particle["radius"])
605
606     # Loại bỏ hạt nếu hết tuổi thọ
607     if particle["lifetime"] <= 0:
608         particles.remove(particle)
609
610     pygame.display.update()
611     pygame.time.delay(30) # Tạo độ trễ mượt mà

```

Hình 19.1. Hàm *fireworks_effect*

- + **surface.fill((0, 0, 0)):** Làm sạch màn hình (xóa sạch màn hình với màu đen).
- + **for particle in particles[:]:** Lặp qua từng hạt pháo hoa để cập nhật vị trí và tuổi thọ.
- + **particle["x"] += particle["vx"]; particle["y"] += particle["vy"]:** Cập nhật vị trí của hạt pháo hoa.
- + **particle["lifetime"] -= 1:** Giảm tuổi thọ của hạt pháo hoa.
- + **pygame.draw.circle(surface, particle["color"], (int(particle["x"]), int(particle["y"])), particle["radius"]):** Vẽ hạt pháo hoa lên màn hình.
- + **if particle["lifetime"] <= 0: particles.remove(particle):** Loại bỏ hạt pháo hoa nếu hết tuổi thọ.
- + **pygame.display.update():** Cập nhật màn hình để hiển thị pháo hoa.
- + **pygame.time.delay(30):** Tạo độ trễ ngắn (30 milliseconds) để hiệu ứng trông mượt mà hơn.

3.2.2. Hàm xử lý giao diện

```

256 def draw_text_middle(text, size, color, surface, offset_y=0):
257     font = pygame.font.SysFont('comicans', size, bold=True)
258     label = font.render(text, 1, color)
259
260     surface.blit(label, (top_left_x + play_width / 2 - label.get_width() / 2,
261                          top_left_y + play_height / 2 - label.get_height() / 2 + offset_y))
262

```

Hình 20. Hàm *draw_text_midle*

- + **pygame.font.SysFont('comicans', size, bold=True):** Tạo phông chữ 'comicans' với kích thước và kiểu đậm.

- + `font.render(text, 1, color)`: Tạo đối tượng hình ảnh từ văn bản với màu sắc đã cho.
- + `surface.blit(...)`: Vẽ văn bản lên bề mặt (surface) tại vị trí căn giữa theo chiều ngang và chiều dọc màn hình.

```

264 def draw_grid(surface, row, col):
265     sx = top_left_x
266     sy = top_left_y
267     for i in range(row):
268         pygame.draw.line(surface, (128,128,128), (sx, sy + i*30), (sx + play_width, sy + i * 30)) # horizontal lines
269         for j in range(col):
270             pygame.draw.line(surface, (128,128,128), (sx + j * 30, sy), (sx + j * 30, sy + play_height)) # vertical lines

```

Hình 21. Hàm `draw_grid`

- + `sx = top_left_x, sy = top_left_y`: Thiết lập tọa độ x và y của góc trên cùng bên trái của lưới.
- + `pygame.draw.line(...)`: Vẽ các đường kẻ ngang và dọc tạo thành các ô trong lưới.

```

272 def draw_exit_button(surface):
273     font = pygame.font.SysFont('comicans', 30, bold=True)
274     label_text = 'Exit'
275     label = font.render(label_text, 1, (0, 0, 0)) # Chữ màu đen
276     exit_x, exit_y = s_width - 120, 70 # Vị trí góc phải trên cùng (dưới nút Pause)
277     button_width, button_height = 100, 40
278
279     mouse_pos = pygame.mouse.get_pos()
280     is_hovered = button_hover(exit_x, exit_y, button_width, button_height, mouse_pos)
281
282     # Thay đổi màu sắc nút khi hover
283     button_color = (255, 120, 120) if is_hovered else (255, 70, 70)
284     border_color = (255, 255, 255) if is_hovered else (200, 200, 200)
285
286     # Vẽ hình chữ nhật bo góc
287     pygame.draw.rect(surface, button_color, (exit_x, exit_y, button_width, button_height), border_radius=10)
288     pygame.draw.rect(surface, border_color, (exit_x, exit_y, button_width, button_height), 2, border_radius=10)
289
290     # Hiển thị chữ trên nút
291     label_x = exit_x + (button_width - label.get_width()) // 2
292     label_y = exit_y + (button_height - label.get_height()) // 2
293     surface.blit(label, (label_x, label_y))
294
295     return exit_x, exit_y, button_width, button_height

```

Hình 22. Hàm `draw_exit_button`

- + `pygame.font.SysFont('comicans', 30, bold=True)`: Tạo phông chữ 'comicans' với kích thước 30 và kiểu đậm.
- + `font.render(label_text, 1, (0, 0, 0))`: Tạo đối tượng hình ảnh từ văn bản "Exit" với màu đen.
- + `pygame.mouse.get_pos()`: Lấy vị trí hiện tại của chuột.
- + `button_hover(...)`: Kiểm tra nếu chuột đang di chuột qua nút.
- + `pygame.draw.rect(...)`: Vẽ hình chữ nhật cho nút với màu nền và màu viền.
- + `surface.blit(label, (label_x, label_y))`: Vẽ văn bản "Exit" lên nút.

```

324 def draw_next_shape(shape, surface):
325     font = pygame.font.SysFont('comicsans', 30)
326     label = font.render('Next Shape', 1, (255,255,255))
327
328     sx = top_left_x + play_width + 50
329     sy = top_left_y + play_height/2 - 100
330     format = shape.shape[shape.rotation % len(shape.shape)]
331
332     for i, line in enumerate(format):
333         row = list(line)
334         for j, column in enumerate(row):
335             if column == '0':
336                 pygame.draw.rect(surface, shape.color, (sx + j*30, sy + i*30, 30, 30), 0)
337
338     surface.blit(label, (sx + 10, sy- 30))

```

Hình 23. Hàm `draw_next_shape`

- + `pygame.font.SysFont('comicsans', 30)`: Tạo phông chữ 'comicsans' với kích thước 30.
- + `font.render('Next Shape', 1, (255,255,255))`: Tạo đối tượng hình ảnh từ văn bản "Next Shape" với màu trắng.
- + `surface.blit(label, (sx + 10, sy- 30))`: Hiển thị nhãn "Next Shape" lên màn hình.
- + `pygame.draw.rect(...)`: Vẽ hình chữ nhật cho khối tiếp theo với màu sắc tương ứng.

```

340 def draw_score(surface):
341     font = pygame.font.SysFont('comicsans', 30)
342     label = font.render(f'Score: {score}', 1, (255, 255, 255))
343
344     sx = top_left_x + play_width + 50 # Vị trí bảng điểm bên trái lưới
345     sy = top_left_y + play_height / 2 + 100
346     surface.blit(label, (sx, sy))

```

Hình 24. Hàm `draw_core`

- + `pygame.font.SysFont('comicsans', 30)`: Tạo phông chữ 'comicsans' với kích thước 30.
- + `font.render(f'Score: {score}', 1, (255, 255, 255))`: Tạo đối tượng hình ảnh từ văn bản điểm số với màu trắng.
- + `sx = top_left_x + play_width + 50`: Đặt vị trí x của bảng điểm bên trái lưới.
- + `sy = top_left_y + play_height / 2 + 100`: Đặt vị trí y của bảng điểm bên trái lưới.
- + `surface.blit(label, (sx, sy))`: Hiển thị nhãn điểm số lên màn hình.

```

353 def draw_pause_message(surface):
354     if paused:
355         draw_text_middle("Paused", 40, (255, 255, 255), surface)
356         pygame.display.update()

```

Hình 25. Hàm `draw_pause_message`

- + **if paused:** Kiểm tra xem trò chơi có đang tạm dừng không.
- + **draw_text_middle("Paused", 40, (255, 255, 255), surface):** Hiển thị thông báo "Paused" với kích thước 40 và màu trắng.
- + **pygame.display.update():** Cập nhật màn hình để hiển thị thông báo.

```

359 def draw_pause_button(surface):
360     font = pygame.font.SysFont('comicans', 30, bold=True)
361     label_text = 'Resume' if paused else 'Pause'
362     label = font.render(label_text, 1, (0, 0, 0)) # Chữ màu đen
363     pause_x, pause_y = s_width - 120, 20 # Vị trí góc phải trên cùng
364     button_width, button_height = 100, 40
365
366     mouse_pos = pygame.mouse.get_pos()
367     is_hovered = button_hover(pause_x, pause_y, button_width, button_height, mouse_pos)
368
369     # Thay đổi màu sắc nút khi hover
370     button_color = (186, 255, 201) if is_hovered else (150, 235, 170)
371     border_color = (255, 255, 255) if is_hovered else (200, 200, 200)
372
373     # Vẽ hình chữ nhật bo góc
374     pygame.draw.rect(surface, button_color, (pause_x, pause_y, button_width, button_height), border_radius=10)
375     pygame.draw.rect(surface, border_color, (pause_x, pause_y, button_width, button_height), 2, border_radius=10)
376
377     # Hiển thị chữ trên nút
378     label_x = pause_x + (button_width - label.get_width()) // 2
379     label_y = pause_y + (button_height - label.get_height()) // 2
380     surface.blit(label, (label_x, label_y))
381
382     return pause_x, pause_y, button_width, button_height

```

Hình 26. Hàm `draw_pause_button`

- + **pygame.font.SysFont('comicans', 30, bold=True):** Tạo phông chữ 'comicans' với kích thước 30 và kiểu đậm.
- + **label_text = 'Resume' if paused else 'Pause':** Chuyển đổi nhãn nút giữa "Resume" và "Pause" dựa trên trạng thái tạm dừng.
- + **label = font.render(label_text, 1, (0, 0, 0)):** Tạo đối tượng hình ảnh từ văn bản với màu đen.
- + **pause_x, pause_y = s_width - 120, 20:** Vị trí góc phải trên cùng.
- + **button_width, button_height = 100, 40:** Kích thước nút.
- + **mouse_pos = pygame.mouse.get_pos():** Lấy vị trí hiện tại của chuột.
- + **is_hovered = button_hover(pause_x, pause_y, button_width, button_height, mouse_pos):** Kiểm tra nếu chuột đang di chuột qua nút.

- + **button_color = (186, 255, 201) if is_hovered else (150, 235, 170)**: Thay đổi màu sắc nút khi hover.
- + **border_color = (255, 255, 255) if is_hovered else (200, 200, 200)**: Thay đổi màu sắc viền nút khi hover.
- + **pygame.draw.rect(surface, button_color, (pause_x, pause_y, button_width, button_height), border_radius=10)**: Vẽ hình chữ nhật bo góc cho nút với màu nền.
- + **pygame.draw.rect(surface, border_color, (pause_x, pause_y, button_width, button_height), 2, border_radius=10)**: Vẽ viền cho nút với màu viền.
- + **label_x = pause_x + (button_width - label.get_width()) // 2**: Căn giữa nhãn theo chiều ngang trên nút.
- + **label_y = pause_y + (button_height - label.get_height()) // 2**: Căn giữa nhãn theo chiều dọc trên nút.
- + **surface.blit(label, (label_x, label_y))**: Hiển thị nhãn lên nút.
- + **return pause_x, pause_y, button_width, button_height**: Trả về vị trí và kích thước của nút.

```

385 def draw_window(surface, background):
386     # Hiển thị ảnh nền nếu có
387     if background:
388         surface.blit(background, (0, 0))
389     else:
390         surface.fill((0, 0, 0)) # Nếu không có ảnh nền, sử dụng nền đen
391
392     # Hiển thị tiêu đề Tetris
393     font = pygame.font.SysFont('comicsans', 60)
394     label = font.render('TETRIS', 1, (255, 255, 255))
395     surface.blit(label, (top_left_x + play_width / 2 - (label.get_width() / 2), 30))
396
397     # Hiển thị các khối trong lưới
398     for i in range(len(grid)):
399         for j in range(len(grid[i])):
400             pygame.draw.rect(surface, grid[i][j], (top_left_x + j * 30, top_left_y + i * 30, 30, 30), 0)
401
402     # Vẽ lưới và viền
403     draw_grid(surface, 20, 10)
404     pygame.draw.rect(surface, (255, 0, 0), (top_left_x, top_left_y, play_width, play_height), 5)
405
406     # Hiển thị bảng điểm và số lần phá hủy
407     draw_score(surface)
408     draw_destroy_count(surface)

```

Hình 27. Hàm `draw_window`

- + **surface.blit(background, (0, 0))**: Vẽ hình nền lên bề mặt.
- + **surface.fill((0, 0, 0))**: Làm đen toàn bộ màn hình nếu không có hình nền.

- + `pygame.font.SysFont('comicans', 60)`: Tạo phông chữ 'comicans' với kích thước 60.
- + `font.render('TETRIS', 1, (255, 255, 255))`: Tạo đối tượng hình ảnh từ văn bản "TETRIS" với màu trắng.
- + `surface.blit(...)`: Vẽ văn bản "TETRIS" lên bề mặt.
- + `pygame.draw.rect(...)`: Vẽ các khối trong lưới và các đường kẻ lưới.
- + `draw_score(surface)`: Hiển thị điểm số.
- + `draw_destroy_count(surface)`: Hiển thị số lần phá hủy.

```

416 def draw_button(surface, label, x, y, width, height, mouse_pos):
417     color = (255, 255, 255) # Màu mặc định
418     if button_hover(x, y, width, height, mouse_pos):
419         color = (200, 200, 200) # Màu khi hover
420     pygame.draw.rect(surface, color, (x, y, width, height))
421     surface.blit(label, (x + 10, y + 10))

```

Hình 28. Hàm `draw_button`

- + `color = (255, 255, 255) # Màu mặc định`: Đặt màu mặc định của nút bấm là màu trắng.
- + `if button_hover(x, y, width, height, mouse_pos)`: Kiểm tra nếu con trỏ chuột đang hover (đi qua) nút bấm.
- + `color = (200, 200, 200) # Màu khi hover`: Nếu con trỏ chuột đi qua nút bấm, thay đổi màu của nút thành màu xám.
- + `pygame.draw.rect(surface, color, (x, y, width, height))`: Vẽ một hình chữ nhật đại diện cho nút bấm với màu đã xác định.
- + `surface.blit(label, (x + 10, y + 10))`: Vẽ (blit) nhãn của nút bấm lên bề mặt (surface) tại vị trí lệch 10 pixel về phía phải và phía dưới từ góc trên bên trái của nút bấm.

```

547 def draw_destroy_count(surface):
548     font = pygame.font.SysFont('comicans', 30)
549
550     # Hiển thị số lần phá hủy còn lại
551     destroy_left_label = font.render(f"Destroy Left: {max_destroy - destroy_count}", True, (255, 255, 255))
552     sx = top_left_x - 235 # Đặt ở bên trái lưới (giảm x để chuyển sang trái)
553     sy = top_left_y + 200 # Đặt dưới phân "TETRIS" một khoảng hợp lý
554     surface.blit(destroy_left_label, (sx, sy))
555
556     # Hiển thị số lần phá hủy tối đa
557     max_destroy_label = font.render(f"Max Destroy: {max_destroy}", True, (255, 255, 255))
558     surface.blit(max_destroy_label, (sx, sy + 40)) # Dịch xuống 40 pixel để không bị đè
559

```

Hình 29. Hàm `draw_destroy_count`

- + **pygame.font.SysFont('comicsans', 30)**: Tạo phông chữ 'comicsans' với kích thước 30.
- + **font.render(f'Destroy Left: {max_destroy - destroy_count}', True, (255, 255, 255))**: Tạo đối tượng hình ảnh từ văn bản "Destroy Left" với màu trắng.
- + **sx = top_left_x - 235**: Đặt tọa độ x của văn bản ở bên trái lưới.
- + **sy = top_left_y + 200**: Đặt tọa độ y của văn bản dưới phần "TETRIS".
- + **surface.blit(destroy_left_label, (sx, sy))**: Vẽ văn bản "Destroy Left" lên bề mặt tại vị trí xác định.
- + **font.render(f'Max Destroy: {max_destroy}', True, (255, 255, 255))**: Tạo đối tượng hình ảnh từ văn bản "Max Destroy" với màu trắng.
- + **surface.blit(max_destroy_label, (sx, sy + 40))**: Vẽ văn bản "Max Destroy" lên bề mặt, dịch xuống 40 pixel để không bị đè lên văn bản khác.

3.3. Quy trình hoạt động

3.3.1. Thiết lập cấu hình ban đầu cho ứng dụng

Import các thư viện cần thiết của ứng dụng:

```
1 import pygame
2 import random
3 import math
4 from pathlib import Path
```

Hình 29. Các thư viện

pygame: Một thư viện phổ biến dùng để phát triển trò chơi 2D trong Python, cung cấp các công cụ xử lý đồ họa, âm thanh và sự kiện, dùng thư viện này để vẽ màn hình, điều khiển khối rơi, xử lý sự kiện như nhấn phím hoặc chuột.

random: Cung cấp các hàm để tạo số ngẫu nhiên. Thư viện này có thể được dùng để chọn ngẫu nhiên các khối (ví dụ: S, Z, I, O, J, L, T).

math: Thư viện toán học cung cấp các hàm như floor, ceil, và các phép toán nâng cao. Dùng để tính toán vị trí hoặc xử lý tọa độ trong trò chơi.

pathlib.Path: Cung cấp cách xử lý đường dẫn tệp. Dùng nó để xây dựng đường dẫn tới file Rank.txt, giúp mã hoạt động đúng trên các hệ điều hành khác nhau.

```
current_directory = Path(__file__).parent
file_path = current_directory / "Rank.txt"
```

Hình 30. Đường dẫn tệp

`Path(__file__).parent`: Lấy đường dẫn thư mục chứa file Python hiện tại.

`file_path`: Tạo đường dẫn đầy đủ tới file `Rank.txt`. File này dùng để lưu xếp hạng hoặc điểm cao của người chơi.

```
9
10  pygame.font.init()
11
```

Hình 31. Khởi tạo Pygame

Dòng này khởi tạo module font của pygame, cho phép bạn hiển thị chữ (ví dụ: điểm số, tên người chơi).

```
12  s_width = 800
13  s_height = 700
14  play_width = 300
15  play_height = 600
16  block_size = 30
17  difficulty = "Easy"
18  fall_speed = 0.5
19  paused = False
20  player_name = ""
21  destroy_count = 0
22  max_destroy = 3
23
24  top_left_x = (s_width - play_width) // 2
25  top_left_y = s_height - play_height
26
27  score = 0
```

Hình 32. Khởi tạo thông số toàn cục

- Kích thước màn hình và lưới
- + `s_width, s_height`: Kích thước cửa sổ trò chơi (800x700).

- + `play_width, play_height`: Kích thước khu vực chơi chính (300x600), chia thành lưới 10x20 ô vuông.
- + `block_size`: Kích thước mỗi ô vuông trong lưới là 30x30 pixel.
 - Trạng thái trò chơi
- + `difficulty`: Độ khó mặc định là "Easy". Có thể dùng để thay đổi tốc độ rơi hoặc thêm tính năng.
- + `fall_speed`: Tốc độ rơi mặc định của các khối là 0.5.
- + `paused`: Kiểm soát trạng thái tạm dừng của trò chơi.
 - Thông tin người chơi
- + `player_name`: Chuỗi lưu tên người chơi.
- + `destroy_count`: Theo dõi số lần người chơi sử dụng "phá hủy".
- + `max_destroy`: Giới hạn số lần "phá hủy" là 3.
 - Tọa độ khu vực chơi
- + `top_left_x, top_left_y`: Tọa độ của góc trên bên trái khu vực chơi, dùng để căn giữa khu vực này trong cửa sổ trò chơi.
 - Điểm số
- + `score`: Điểm số của người chơi, mặc định là 0.

```

28 background_path = Path(__file__).parent / "AnhNen1.png" # Đường dẫn tới ảnh nền
29 try:
30     background = pygame.image.load(str(background_path))
31     background = pygame.transform.scale(background, (s_width, s_height)) # Điều chỉnh kích thước ảnh nền
32 except pygame.error as e:
33     print(f"Lỗi khi tải ảnh nền: {e}")
34     background = None # Nếu không có ảnh nền, trò chơi sẽ tiếp tục mà không có nền

```

Hình 33. Tải ảnh nền

`background_path`: Tạo đường dẫn tới file ảnh nền AnhNen1.png trong cùng thư mục với file Python.

`pygame.image.load`: Tải ảnh từ đường dẫn.

`pygame.transform.scale`: Thay đổi kích thước ảnh nền sao cho vừa với kích thước cửa sổ trò chơi (`s_width, s_height`).

Xử lý lỗi: Nếu xảy ra lỗi khi tải ảnh (ví dụ: file không tồn tại), in thông báo lỗi và đặt giá trị `background` là `None`.

```

37 # Khởi tạo hệ thống âm thanh
38 pygame.mixer.init()
39

```

Hình 34. Khởi tạo hệ thống âm thanh

`pygame.mixer.init()`: Khởi tạo module mixer của pygame, cho phép xử lý âm thanh và nhạc nền.

```

41 music_path = Path(__file__).parent / "NhacNen.mp3"
42
43 try:
44     pygame.mixer.music.load(str(music_path))
45     pygame.mixer.music.play(-1)
46     pygame.mixer.music.set_volume(0.5)
47 except pygame.error as e:
48     print(f"Lỗi khi phát nhạc: {e}")

```

Hình 35. Phát nhạc nền

- + `music_path`: Đường dẫn tới file nhạc nền `NhacNen.mp3`.
- + `pygame.mixer.music.load`: Tải file nhạc nền từ đường dẫn.
- + `pygame.mixer.music.play(-1)`: Phát nhạc lặp lại vô hạn. Nếu muốn phát một lần, thay -1 bằng 0.
- + `pygame.mixer.music.set_volume(0.5)`: Điều chỉnh âm lượng nhạc nền. Giá trị nằm trong khoảng từ 0.0 (im lặng) đến 1.0 (to nhất).

Xử lý lỗi: Nếu xảy ra lỗi (ví dụ: file không tồn tại hoặc định dạng không hỗ trợ), in thông báo và bỏ qua phần phát nhạc.

```

51 try:
52     button_click_sound = pygame.mixer.Sound(str(Path(__file__).parent / "NhacBam.wav"))
53     button_click_sound.set_volume(0.5)
54 except pygame.error as e:
55     print(f"Lỗi khi tải âm thanh: {e}")
56     button_click_sound = None

```

Hình 36. Âm thanh của nút bấm

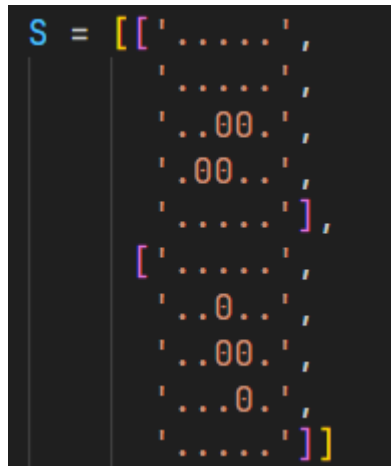
- + `pygame.mixer.Sound`: Tải file âm thanh nút bấm `NhacBam.wav`.
- + `set_volume(0.5)`: Điều chỉnh âm lượng âm thanh nút bấm, tương tự như nhạc nền.

Xử lý lỗi: Nếu xảy ra lỗi (file không tồn tại hoặc định dạng không hỗ trợ), in thông báo và đặt giá trị button_click_sound là None.

Khởi tạo các hình khối Tetris:

Các hình khối được mô tả bằng các mảng 2 chiều, mỗi mảng đại diện cho một hình khối trong trò chơi. Các ký tự '.' đại diện cho không gian trống, và 0 đại diện cho phần của khối.

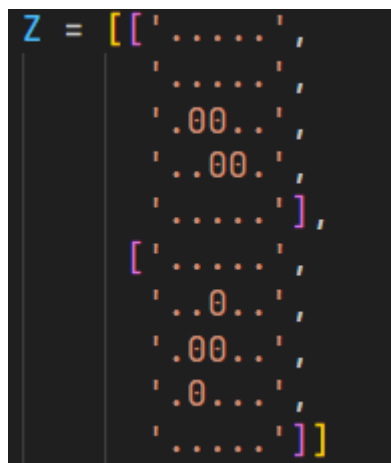
- Khối S



Hình 37. Khối S

Khối S có 2 trạng thái xoay

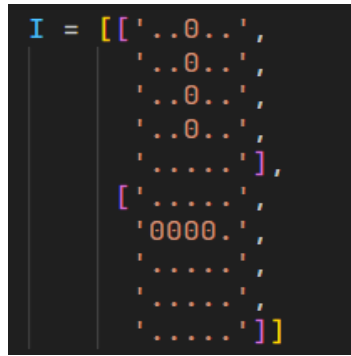
- Khối Z



Hình 38. Khối Z

Khối Z có 2 trạng thái xoay

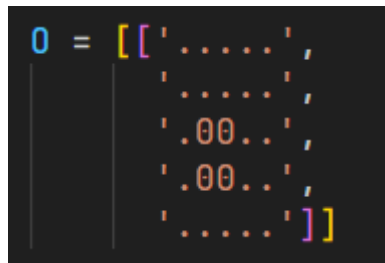
- Khối I



Hình 39. Khối I

Khối I có 2 trạng thái xoay

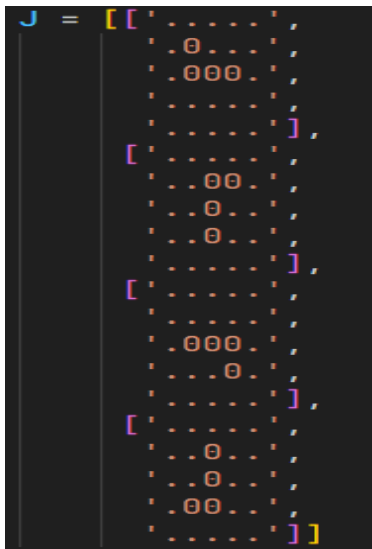
- Khối O



Hình 40. Khối O

Khối O có 1 trạng thái xoay

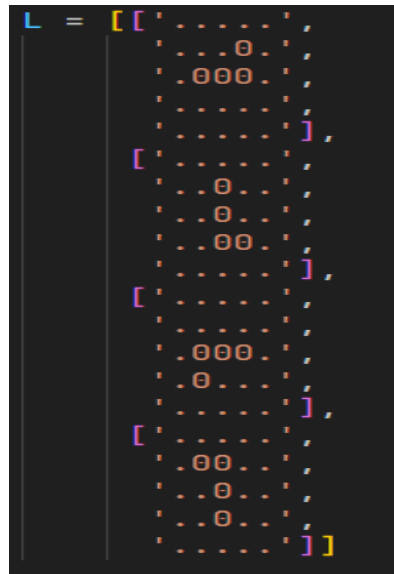
- Khối J



Hình 41. Khối J

Khối I có 4 trạng thái xoay

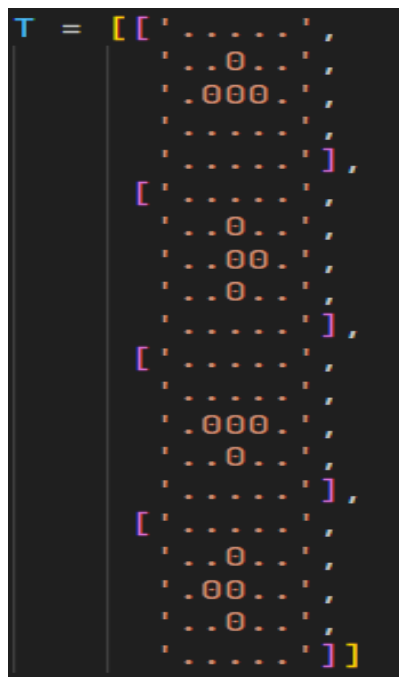
- Khối L



Hình 42. Khối L

Khối L có 4 trạng thái xoay

- Khối T



Hình 43. Khối T

Khối T có 2 trạng thái xoay

Danh sách các Shapes:

```
shapes = [S, Z, I, O, J, L, T]
```

Hình 44. Danh sách các Shapes

Danh sách shapes chứa tất cả các hình khối Tetris, mỗi phần tử là một trong các hình khối đã khai báo ở trên. Khi trò chơi chạy, bạn sẽ chọn ngẫu nhiên một hình khối từ danh sách này để rơi xuống màn hình.

Danh sách màu của các Shapes:

```
163 shape_colors = [(0, 255, 0), (255, 0, 0), (0, 255, 255), (255, 255, 0), (255, 165, 0), (0, 0, 255), (128, 0, 128)]
164 # index 0 - 6 represent shape
```

Hình 45. Danh sách màu của các Shapes

Danh sách này chứa các giá trị RGB cho mỗi hình khối:

(0, 255, 0) là màu xanh lá cây (cho khối S).

(255, 0, 0) là màu đỏ (cho khối Z).

(0, 255, 255) là màu cyan (cho khối I).

(255, 255, 0) là màu vàng (cho khối O).

(255, 165, 0) là màu cam (cho khối J).

(0, 0, 255) là màu xanh dương (cho khối L).

(128, 0, 128) là màu tím (cho khối T).

3.3.2. Cách thức hoạt động

Mục đích của phần này là phân tích cách hoạt động của ứng dụng thông qua các hàm như `main()`, `main_menu()`, `setting_menu()`, `display_rankings()`.

Hàm `main_menu()`

Hàm này sẽ tương tác với người dùng trước khi người dùng chơi trò chơi.

```
def main_menu():
    # Nếu nhạc đang tạm dừng (pause), tiếp tục phát
    if not pygame.mixer.music.get_busy():
        pygame.mixer.music.unpause()
    # Tải ảnh nền
    background_path = Path(__file__).parent / "AnhNen8.png" # Đảm bảo file AnhNen.png nằm cùng thư mục với script
    background = pygame.image.load(str(background_path))
    background = pygame.transform.scale(background, (s_width, s_height)) # Thay đổi kích thước ảnh cho vừa màn hình
```

Hình 46. Hàm `main_menu`

Chức năng của đoạn mã trên là tải hình ảnh lên từ file đã có trong thư mục, ta đến với vòng lặp chính của hàm.

```
run = True
while run:
    win.blit(background, (0, 0)) # Vẽ ảnh nền lên màn hình
```

Hình 46.1. Hàm `main_menu`

Vòng lặp này sẽ tiếp tục chạy cho đến khi người chơi thoát khỏi menu. Mỗi lần lặp, nó sẽ vẽ lại giao diện menu, hiển thị hình nền, các nút, và các thông tin liên quan đến độ khó.

```
# Tạo các nút menu với hiệu ứng viền
font_menu = pygame.font.SysFont('comicsans', 50, bold=True)
button_spacing = 80 # Khoảng cách giữa các nút
start_y = 300 # Điểm bắt đầu cho các nút
button_width = 350 # Chiều rộng của nút (tăng lên so với 300)
button_height = 60 # Chiều cao của nút (tăng lên so với 50)

# Danh sách nút (text, vị trí Y)
buttons = [
    ("Play", start_y),
    ("GameMode", start_y + button_spacing),
    ("Ranking", start_y + button_spacing * 2),
    ("Quit", start_y + button_spacing * 3),
]
```

Hình 46.2. Hàm `main_menu`

Tạo các phần tử của giao diện. Mỗi nút đại diện cho 1 chức năng khi nhấn vào bằng chuột.

```
for button_text, y_pos in buttons:
    # Kiểm tra nếu chuột đang hover nút
    is_hovered = button_hover(s_width // 2 - button_width // 2, y_pos, button_width, button_height, mouse_pos)

    button_color = (255, 255, 255) if not is_hovered else (173, 216, 230) # Màu nền
    border_color = (0, 0, 0) if is_hovered else (0, 0, 0) # Màu viền

    # Vẽ hình chữ nhật cho nút (có viền)
    pygame.draw.rect(win, button_color, (s_width // 2 - button_width // 2, y_pos, button_width, button_height)) # Nền nút
    pygame.draw.rect(win, border_color, (s_width // 2 - button_width // 2, y_pos, button_width, button_height), 3) # Viền nút

    # Vẽ text chính giữa ô
    label = font_menu.render(button_text, 1, (0, 0, 0) if not is_hovered else (0, 0, 0)) # Màu text
    label_x = s_width // 2 - label.get_width() // 2 # Căn giữa theo chiều ngang
    label_y = y_pos + (button_height - label.get_height()) // 2 # Căn giữa theo chiều dọc
    win.blit(label, (label_x, label_y))
```

Hình 46.3. Hàm `main_menu`

Hiệu ứng hover: Khi người chơi di chuột qua một nút, nút đó sẽ đổi màu, tạo cảm giác tương tác.

Màu sắc nút: Mỗi nút sẽ có màu khác khi nó đang được chọn (màu xanh matcha), khi di chuột qua (màu xanh nhạt), hoặc mặc định (màu trắng).

Vẽ nút: Dùng `pygame.draw.rect` để vẽ các nút với màu nền, viền và độ bo góc.

```

# Xử lý sự kiện
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.mixer.music.stop() #####
        run = False
        pygame.quit()
        quit()

    if event.type == pygame.MOUSEBUTTONDOWN:
        if button_hover(s_width // 2 - button_width // 2, start_y, button_width, button_height, mouse_pos):
            if button_click_sound:
                button_click_sound.play() # Phát âm thanh khi nhấn nút
                enter_player_name() # Yêu cầu nhập tên
                main() # Bắt đầu trò chơi
            elif button_hover(s_width // 2 - button_width // 2, start_y + button_spacing, button_width, button_height, mouse_pos):
                if button_click_sound:
                    button_click_sound.play() # Phát âm thanh khi nhấn nút
                    settings_menu() # Chuyển tới menu cài đặt
            elif button_hover(s_width // 2 - button_width // 2, start_y + button_spacing * 2, button_width, button_height, mouse_pos):
                if button_click_sound:
                    button_click_sound.play() # Phát âm thanh khi nhấn nút
                    display_rankings() # Hiển thị bảng xếp hạng
            elif button_hover(s_width // 2 - button_width // 2, start_y + button_spacing * 3, button_width, button_height, mouse_pos):
                if button_click_sound:
                    button_click_sound.play() # Phát âm thanh khi nhấn nút
                run = False # Thoát game

```

Hình 46.4. Hàm `main_menu`

Lắng nghe sự kiện diễn ra và xử lý nhấp chuột:

- Sự kiện thoát `pygame.QUIT`
- + Đóng cửa sổ: Khi người chơi nhấn vào nút đóng cửa sổ, nhạc nền sẽ dừng và trò chơi sẽ thoát.
- Sự kiện nhấp chuột
- + Play: Khi người chơi nhấn vào nút "Play", hàm:
Gọi `enter_player_name()` để yêu cầu người chơi nhập tên.
Gọi `main()` để bắt đầu trò chơi.
- + GameMode: Khi nhấn nút "GameMode", chuyển đến menu cài đặt độ khó bằng `settings_menu()`.
- + Ranking: Hiển thị bảng xếp hạng với `display_rankings()`.
- + Quit: Khi nhấn vào nút "Quit", vòng lặp menu kết thúc (`run = False`), thoát trò chơi.

```

def enter_player_name():
    global player_name
    run = True
    input_box_width = 300 # Chiều rộng khung nhập
    input_box_height = 50 # Chiều cao khung nhập
    input_box = pygame.Rect(s_width // 2 - input_box_width // 2, s_height // 2 - input_box_height // 2, input_box_width, input_box_height)

    font = pygame.font.SysFont('comicans', 40)
    player_name = ""

    # Tải ảnh nền (nếu có)
    background_path = Path(__file__).parent / "AnhNen1.png"
    try:
        background = pygame.image.load(str(background_path))
        background = pygame.transform.scale(background, (s_width, s_height)) # Thay đổi kích thước ảnh phù hợp
    except pygame.error as e:
        print(f"Lỗi khi tải hình ảnh: {e}")
        background = None

```

Hình 47. Hàm `enter_player_name`

Hàm này được gọi ra khi người chơi nhấn chuột vào nút “Play”

Xác định được kích thước và vị trí của khung nhập (input box): Tạo một hình chữ nhật đại diện cho khung nhập và căn giữa màn hình.

Sử dụng font chữ ‘comicans’ để hiển thị nội dung nhập.

Tạo thêm biến để lưu tên người chơi.

```
while run:
    # Hiển thị ảnh nền nếu tồn tại
    if background:
        win.blit(background, (0, 0))
    else:
        win.fill((255, 255, 255)) # Làm sạch màn hình (nền trắng nếu không có ảnh nền)

    # Hiển thị tiêu đề với màu trắng
    label = font.render("Enter your name:", True, (255, 255, 255)) # Chữ màu trắng
    win.blit(label, (s_width // 2 - label.get_width() // 2, input_box.y - 60))

    # Vẽ nền và viền khung nhập
    pygame.draw.rect(win, (255, 255, 255), input_box) # Nền khung màu trắng
    pygame.draw.rect(win, (0, 0, 0), input_box, 2) # Viền khung màu đen

    # Hiển thị văn bản bên trong khung nhập
    text_surface = font.render(player_name, True, (0, 0, 0)) # Chữ màu đen
    text_x = input_box.x + 10 # Thụt vào từ lề trái
    text_y = input_box.y + (input_box.height - text_surface.get_height()) // 2 # Căn giữa dọc
    win.blit(text_surface, (text_x, text_y))

    pygame.display.flip()

    # Lắng nghe sự kiện
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
            pygame.quit()
            quit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN: # Nhấn Enter để xác nhận tên
                if player_name.strip(): # Chỉ chấp nhận nếu có tên
                    return player_name
            elif event.key == pygame.K_BACKSPACE: # Xóa ký tự cuối cùng
                player_name = player_name[:-1]
            else:
                if len(player_name) < 13: # Giới hạn tên tối đa 13 ký tự
                    player_name += event.unicode # Thêm ký tự vào tên
```

Hình 47.1. Hàm `enter_player_name`

Đây là vòng lặp chính của hàm:

- Vẽ giao diện:
- + Hiển thị khung nhập, nhãn hướng dẫn, và tên người chơi.
- + Khung nhập có viền đổi màu khi chuột di chuyển qua.
- Xử lý sự kiện:
- + Nếu người dùng nhấn phím:

- + Nhập ký tự Unicode vào chuỗi `player_name` (giới hạn tối đa 13 ký tự).
- + Xóa ký tự cuối cùng khi nhấn Backspace.
- + Kết thúc vòng lặp nếu nhấn Enter và tên hợp lệ.
- + Nếu người dùng đóng cửa sổ, thoát khỏi trò chơi.

Hàm `main()`:

Hàm `main` đóng vai trò trung tâm trong trò chơi Tetris, điều khiển toàn bộ hoạt động từ xử lý khối (rơi, xoay, va chạm), giao diện người dùng (nút bấm, hiệu ứng), đến lưu trữ và hiển thị điểm số. Nó liên kết các thành phần và đảm bảo tính liên tục trong trò chơi.

- Khởi tạo

```
global grid, paused, fall_speed, destroy_count, score
destroy_count = 0
score = 0
if not pygame.mixer.music.get_busy():
    pygame.mixer.music.unpause()

locked_positions = {} # Lưu các vị trí đã khóa của khối
grid = create_grid(locked_positions)

change_piece = False
run = True
current_piece = get_shape()
next_piece = get_shape()
clock = pygame.time.Clock()
fall_time = 0
```

Hình 48. Hàm main

Tải nhạc nền và tiếp tục phát nếu bị tạm dừng.

Tạo lưới và khối đầu tiên để bắt đầu trò chơi.

Thiết lập biến `clock` để kiểm soát thời gian và tốc độ rơi (`fall_speed`).

- Vòng lặp chính
- + Tính toán thời gian rơi:

```
while run:
    grid = create_grid(locked_positions)
    fall_time += clock.get_rawtime()
    clock.tick()
```

Hình 48.1. Hàm main

Sử dụng thời gian thực từ clock để điều khiển tốc độ rơi của khối.

+ Xử lý sự kiện:

Bàn phím:

```
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_p: # Phím 'P' để tạm dừng
        toggle_pause()

    if not paused: # Chỉ xử lý phím khi không bị tạm dừng
        if event.key == pygame.K_LEFT:
            current_piece.x -= 1
            if not valid_space(current_piece, grid):
                current_piece.x += 1

        elif event.key == pygame.K_RIGHT:
            current_piece.x += 1
            if not valid_space(current_piece, grid):
                current_piece.x -= 1

        elif event.key == pygame.K_UP:
            current_piece.rotation = (current_piece.rotation + 1) % len(current_piece.shape)
            if not valid_space(current_piece, grid):
                current_piece.rotation = (current_piece.rotation - 1) % len(current_piece.shape)

        elif event.key == pygame.K_DOWN:
            current_piece.y += 1
            if not valid_space(current_piece, grid):
                current_piece.y -= 1

        elif event.key == pygame.K_SPACE:
            while valid_space(current_piece, grid):
                current_piece.y += 1
            current_piece.y -= 1 # Điều chỉnh vị trí
```

Hình 48.2. Hàm main

Di chuyển khối trái/phải bằng phím LEFT và RIGHT.

Xoay khối bằng phím UP.

Rơi nhanh bằng phím DOWN hoặc thả rơi tự do với SPACE.

Tạm dừng bằng phím P.

Chuột:

```

if event.type == pygame.MOUSEBUTTONDOWN:
    if event.button == 1: # Nút chuột trái
        mouse_pos = pygame.mouse.get_pos() # Lấy vị trí chuột
        destroy_blocks_with_effect_at_mouse(win, locked_positions, mouse_pos) # Gọi hàm phá hủy

if event.type == pygame.MOUSEBUTTONDOWN:
    mouse_pos = pygame.mouse.get_pos()

    # Xử lý nút Exit
    exit_x, exit_y, exit_width, exit_height = draw_exit_button(win)
    if button_hover(exit_x, exit_y, exit_width, exit_height, mouse_pos):
        run = False # Thoát trò chơi

    # Xử lý nút Pause
    pause_x, pause_y, pause_width, pause_height = draw_pause_button(win)
    if button_hover(pause_x, pause_y, pause_width, pause_height, mouse_pos):
        toggle_pause() # Chuyển trạng thái tạm dừng

```

Hình 48.3. Hàm main

Xử lý các nút "Pause" và "Exit" bằng cách kiểm tra vị trí chuột.

Kích hoạt hiệu ứng phá hủy khối tại vị trí nhấp chuột.

+ Cập nhật giao diện:

```

draw_window(win, background)
draw_next_shape(next_piece, win)
draw_pause_button(win)
draw_exit_button(win)
pygame.display.update()

```

Hình 48.4. Hàm main

Hiển thị khối, lưới, nút bấm, khối tiếp theo, và điểm số.

Làm mới màn hình sau mỗi hành động.

- Logic trò chơi

```

# Logic rơi khối
if fall_time / 1000 >= fall_speed:
    fall_time = 0
    current_piece.y += 1
    if not valid_space(current_piece, grid) and current_piece.y > 0:
        current_piece.y -= 1
        change_piece = True

shape_pos = convert_shape_format(current_piece)
for i in range(len(shape_pos)):
    x, y = shape_pos[i]
    if y > -1:
        grid[y][x] = current_piece.color

# Khi khối chạm đáy hoặc khối khác
if change_piece:
    for pos in shape_pos:
        locked_positions[(pos[0], pos[1])] = current_piece.color
    current_piece = next_piece
    next_piece = get_shape()
    change_piece = False

# Kiểm tra nếu người chơi thua
if check_lost(locked_positions):
    run = False # Kết thúc trò chơi

# Xóa các hàng đầy
clear_rows(grid, locked_positions)

```

Hình 48.5. Hàm main

+ Va chạm và khóa khối: Khi khối chạm đáy hoặc va chạm khối khác, nó sẽ được "khóa" vào vị trí và không thể di chuyển.

Tạo khối mới và kiểm tra nếu người chơi thua.

+ Xóa hàng:

Kiểm tra và xóa các hàng đầy đủ, cập nhật điểm số tương ứng.

+ Hiệu ứng:

Tạm dừng nhạc khi tạm dừng trò chơi.

Hiển thị thông báo thua khi trò chơi kết thúc.

- Kết thúc

Sau khi thua:

```

# Lưu điểm và công bố hạng
rank, rankings = save_score_and_get_rank(player_name, score)

# Hiển thị hiệu ứng và thông báo nếu lọt top 5
if rank is not None:
    # Hiển thị ảnh nền trước
    if background: # Kiểm tra nếu ảnh nền đã được tải
        win.blit(background, (0, 0))
    else:
        win.fill((0, 0, 0)) # Nếu không có ảnh nền, sử dụng nền đen

    if rank == 1:
        fireworks_effect_pro(win, num_fireworks=20, duration=3000) # Pháo hoa nhiều hơn cho top 1
        draw_text_middle("Champion!", 60, (255, 255, 0), win)
    elif rank <= 5:
        fireworks_effect(win, num_fireworks=10, duration=2000) # Pháo hoa cho top 5
        draw_text_middle("Top 5!", 40, (255, 255, 0), win)

```

Hình 48.6. Hàm main

Hiển thị điểm số, hạng người chơi, và hiệu ứng (pháo hoa).

```

# Hiển thị bảng xếp hạng nếu có
if rankings:
    if background:
        win.blit(background, (0, 0)) # Hiển thị ảnh nền
    else:
        win.fill((0, 0, 0)) # Nếu không có ảnh nền, dùng nền đen

    title = font.render("Top Rankings:", 1, (255, 255, 255))
    win.blit(title, (s_width // 2 - title.get_width() // 2, 50))

    for i, (name, score_str) in enumerate(rankings[:10], start=1): # Hiển thị top 10
        label = font.render(f"{i}. {name} - {score_str}", 1, (255, 255, 255))
        win.blit(label, (s_width // 2 - label.get_width() // 2, 100 + i * 40))

    pygame.display.update()
    pygame.time.delay(5000)

# Quay lại menu chính
main_menu()

```

Hình 48.7. Hàm main

Cung cấp bảng xếp hạng (nếu có) và chuyển về menu chính.

Hàm setting_menu():

Hàm này sẽ được gọi ra khi người chơi nhấn chuột vào button “GameMode”.

- Khởi tạo:

```

def settings_menu():
    try:
        # Tải hình ảnh
        background_path = Path(__file__).parent / "AnhNen1.png" # Đường dẫn tới
        try:
            background = pygame.image.load(str(background_path))
            background = pygame.transform.scale(background, (s_width, s_height))
        except pygame.error as e:
            print(f"Lỗi khi tải hình ảnh: {e}")
        return

```

Hình 49. Hàm setting_menu

Sử dụng `pygame.image.load` để tải hình nền từ đường dẫn.

Thay đổi kích thước ảnh phù hợp với kích thước cửa sổ trò chơi (`s_width`, `s_height`).

Kiểm tra lỗi khi tải ảnh, nếu có lỗi sẽ thoát và in ra thông báo.

```
while run:
    # Hiển thị hình nền
    win.blit(background, (0, 0))

    # Tạo font và nhãn tiêu đề
    font_title = pygame.font.SysFont('comicsans', 60)
    title_label = font_title.render('Select Difficulty', 1, (255, 255, 255))

    # Tạo nhãn hiển thị độ khó hiện tại
    font_current = pygame.font.SysFont('comicsans', 40)
    current_label = font_current.render(f'Current: {difficulty}', 1, (255, 255, 255))

    # Khoảng cách dọc giữa các thành phần
    spacing = 100

    # Vị trí tiêu đề
    title_y = 50
    title_x = s_width // 2 - title_label.get_width() // 2

    # Vị trí các nhãn
    current_x = s_width // 2 - current_label.get_width() // 2
    current_y = title_y + title_label.get_height() + 20 # Cách tiêu đề một chút

    # Kích thước nút
    button_width, button_height = 200, 60
    button_x = s_width // 2 - button_width // 2

    easy_y = current_y + spacing
    normal_y = easy_y + spacing
    hard_y = normal_y + spacing

    # Tọa độ và kích thước các nút
    buttons = [
        ("Easy", easy_y, "Easy", (0, 128, 0)), # Màu chữ cho Easy
        ("Normal", normal_y, "Normal", (255, 165, 0)), # Màu chữ cho Normal
        ("Hard", hard_y, "Hard", (255, 0, 0)), # Màu chữ cho Hard
    ]
```

Hình 49.1. Hàm `setting_menu`

Cài đặt nút bấm: Tạo ba nút tương ứng với ba mức độ khó: Easy, Normal, Hard. Mỗi nút có màu sắc đặc trưng, kèm theo hiệu ứng khi hover và chọn.

- Vòng lặp chính dùng để xử lý sự kiện:

```
# Lắng nghe sự kiện
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        run = False
        pygame.quit()
        quit()

    if event.type == pygame.MOUSEBUTTONDOWN:
        mouse_pos = pygame.mouse.get_pos()

        # Kiểm tra click vào các nút
        for label, y_pos, value, _ in buttons:
            if button_hover(button_x, y_pos, button_width, button_height, mouse_pos):
                if button_click_sound:
                    button_click_sound.play() # Phát âm thanh khi nhấn nút
                set_difficulty(value)
                run = False
```

Hình 49.2. Hàm setting_menu

Thoát chương trình khi nhấn QUIT.

Xác định hành động khi người chơi nhấn chuột:

Nếu chuột nhấn vào nút bất kỳ, độ khó sẽ được cập nhật.

Phát âm thanh khi bấm nút

Hàm display_rankings():

Hàm display_rankings hiển thị danh sách xếp hạng điểm số cao nhất (đọc từ tệp Rank.txt) trên giao diện trò chơi. Người chơi có thể xem Top 5 người chơi và quay lại menu chính thông qua nút "Back".

```
def display_rankings():
    try:
        # Kiểm tra file hình ảnh
        background_path = Path(__file__).parent / "AnhNen1.png"
        if not background_path.exists():
            print("File hình ảnh không tồn tại:", background_path)
            return
        background = pygame.image.load(str(background_path))
        background = pygame.transform.scale(background, (s_width, s_height))

        # Đọc file Rank.txt
        file_path = Path(__file__).parent / "Rank.txt"
        if not file_path.exists():
            file_path.touch() # Tạo file nếu chưa có
            print("Rank.txt đã được tạo. Hiện không có dữ liệu.")
            win.fill((0, 0, 0))
            draw_text_middle("Rank.txt đã được tạo. Hiện không có dữ liệu.", 40, (255, 255, 255), win)
            pygame.display.update()
            pygame.time.delay(2000)
            return

        with file_path.open(mode="r") as file:
            lines = file.readlines()
            rankings = []
            for line in lines:
                if line.strip():
                    try:
                        name, score = line.strip().split(",")
                        rankings.append((name, int(score)))
                    except ValueError:
                        print(f"Dữ liệu không hợp lệ trong Rank.txt: {line.strip()}")
                        continue

            rankings = sorted(rankings, key=lambda x: x[1], reverse=True)
```

Hình 50. Hàm display_rankings

- Xử lý tệp:

Kiểm tra định dạng tệp Rank.txt:

Các dòng phải có dạng name,score.

Bỏ qua các dòng không hợp lệ và ghi log cảnh báo.

Sắp xếp danh sách rankings theo thứ tự giảm dần dựa trên score.

```

while waiting:
    # Hiển thị ảnh nền
    win.blit(background, (0, 0))

    # Hiển thị tiêu đề
    font_title = pygame.font.SysFont('comicsans', 60)
    title_label = font_title.render('Rankings', 1, (255, 255, 255))
    win.blit(title_label, (s_width // 2 - title_label.get_width() // 2, 50))

    # Hiển thị top 5 người chơi
    font = pygame.font.SysFont('comicsans', 40)
    y_offset = 150 # Khoảng cách từ trên cùng xuống
    for i, (name, score) in enumerate(rankings[:5]): # Hiển thị top 5
        rank_label = font.render(f"#{i + 1}: {name} - {score}", 1, (255, 255, 255))
        win.blit(rank_label, (s_width // 2 - rank_label.get_width() // 2, y_offset))
        y_offset += 50 # Khoảng cách giữa các hàng

    # Hiển thị nút "Back"
    button_width, button_height = 200, 60
    button_x = s_width // 2 - button_width // 2
    button_y = s_height - 150

    # Lấy vị trí chuột
    mouse_pos = pygame.mouse.get_pos()

    # Kiểm tra trạng thái hover
    is_hovered = button_hover(button_x, button_y, button_width, button_height, mouse_pos)

    # Thay đổi màu nút dựa trên trạng thái hover
    button_color = (255, 255, 255) if not is_hovered else (173, 216, 230) # Màu nền
    border_color = (0, 0, 0) if is_hovered else (0, 0, 0) # Màu viền

    pygame.draw.rect(win, button_color, (button_x, button_y, button_width, button_height))
    pygame.draw.rect(win, border_color, (button_x, button_y, button_width, button_height), 3)

    back_label = font.render("Back", 1, (0, 0, 0))
    win.blit(back_label, (button_x + (button_width - back_label.get_width() // 2,

```

Hình 50.1. Hàm `display_rankings`

- Hiển thị dữ liệu:

Hiển thị Top 5 người chơi:

Thông tin dạng "#rank: name - score".

Dữ liệu căn giữa màn hình và cách đều theo chiều dọc.

Hàm `save_score_and_get_rank()`:

Hàm `save_score_and_get_rank` thực hiện hai nhiệm vụ. Lưu điểm của người chơi vào tệp `Rank.txt`. Tính toán thứ hạng của người chơi dựa trên danh sách xếp hạng trong tệp.

- Quy trình hoạt động

```

file_path = Path(__file__).parent / "Rank.txt"

# Lưu điểm của người chơi vào file
try:
    with file_path.open(mode="a") as file:
        file.write(f"{player_name},{score}\n") # Ghi tên và điểm vào file
except Exception as e:
    print(f"Lỗi khi lưu điểm: {e}")
    return None, None

```

Hình 51. Hàm `save_score_and_get_rank`

Ghi điểm vào tệp

Tệp Rank.txt:

Mở tệp ở chế độ "append" (a) để thêm tên và điểm số mới mà không xóa dữ liệu cũ.

Dữ liệu được ghi theo định dạng: player_name,score\n.

Xử lý lỗi:

```
except Exception as e:
    print(f"Lỗi khi xác định hạng: {e}")
    return None, None
```

Hình 51.1. Xử lý lỗi

Nếu xảy ra lỗi trong quá trình ghi (như tệp không tồn tại hoặc không thể mở), thông báo lỗi được in ra và trả về (None, None).

+ Đọc và xác định thứ hạng

```
try:
    with file_path.open(mode="r") as file:
        lines = file.readlines()
```

Hình 51.2. Đọc và xác định thứ hạng

Đọc dữ liệu từ tệp:

Tệp được mở ở chế độ "read" (r), mỗi dòng dữ liệu được đọc vào danh sách lines.

Bỏ qua các dòng trống hoặc không hợp lệ (qua if line.strip()).

Chuyển đổi và sắp xếp:

```
try:
    with file_path.open(mode="r") as file:
        lines = file.readlines()
        rankings = [line.strip().split(",") for line in lines if line.strip()]
        rankings = sorted(rankings, key=lambda x: int(x[1]), reverse=True) # S
```

Hình 51.3. Chuyển đổi và sắp xếp

Mỗi dòng được tách thành hai phần: name và score thông qua split(",").

Danh sách rankings được sắp xếp giảm dần theo score sử dụng hàm sorted.

Xác định thứ hạng:

```
for rank, (name, score_str) in enumerate(rankings, start=1):
    if name == player_name and int(score_str) == score:
        return rank, rankings # Chỉ trả về hạng và bảng xếp
```

Hàm 51.3. Xác định thứ hạng

Lắp qua danh sách xếp hạng (rankings) với chỉ số bắt đầu từ 1 (start=1).

Nếu tìm thấy một mục trùng khớp với player_name và score, trả về thứ hạng của người chơi (rank) và toàn bộ danh sách xếp hạng (rankings).

CHƯƠNG 4: ĐÁNH GIÁ QUÁ TRÌNH HOÀN THÀNH ĐỒ ÁN VÀ GIẢI PHÁP NÂNG CAO HIỆU QUẢ CỦA ỨNG DỤNG GAME TETRIS TRONG THỜI GIAN TỚI

4.1. Đánh giá quá trình hoàn thành đồ án

- Mục tiêu đạt được

- Đồ án đã hoàn thành việc thiết kế và phát triển một phiên bản game Tetris cơ bản với các chức năng chính như: tạo khối, di chuyển, xoay, xếp khối, tính điểm và kết thúc trò chơi.
- Giao diện người dùng thân thiện, dễ thao tác, phù hợp với đối tượng mục tiêu ban đầu.
- Tích hợp các yếu tố cơ bản của lập trình đồ họa và quản lý sự kiện trong quá trình phát triển.

- Những khó khăn gặp phải

- Tối ưu hóa thuật toán xử lý các khối khi xếp hàng: Việc đảm bảo game hoạt động trơn tru ở nhiều cấu hình máy tính khác nhau đã tiêu tốn nhiều thời gian.
- Tích hợp âm thanh và hiệu ứng: Do hạn chế về thời gian, chưa thể hoàn thiện hệ thống âm thanh và các hiệu ứng trực quan nâng cao.
- Phát triển thêm các tính năng mở rộng như chế độ chơi đôi hoặc bảng xếp hạng online gặp khó khăn do hạn chế về kinh nghiệm trong lập trình mạng.

- Nhận xét chung

- Đồ án đã hoàn thành đúng tiến độ và đáp ứng yêu cầu cơ bản đề ra. Tuy nhiên, để đạt hiệu quả cao hơn, cần cải thiện một số khía cạnh kỹ thuật cũng như mở rộng tính năng trong tương lai.

4.2. Giải pháp nhằm nâng cao hiệu quả của ứng dụng game trong thời gian tới

- Tối ưu hóa hiệu năng

- Nâng cấp thuật toán xử lý logic khối, đặc biệt ở khía cạnh kiểm tra và xóa hàng để giảm mức độ tiêu thụ tài nguyên CPU.
- Tích hợp công cụ kiểm thử hiệu năng để đảm bảo game hoạt động mượt mà trên các thiết bị cấu hình thấp.
- **Cải thiện giao diện và trải nghiệm người dùng (UI/UX)**
 - Nâng cấp giao diện với các hiệu ứng động, màu sắc bắt mắt và thân thiện hơn với người dùng.
 - Thêm các tính năng tùy chỉnh như thay đổi chủ đề (theme), điều chỉnh độ khó theo cấp độ chơi.
- **Mở rộng tính năng**
 - Phát triển chế độ chơi nhiều người (multiplayer) qua mạng LAN hoặc Internet để tăng tính tương tác.
 - Tích hợp bảng xếp hạng trực tuyến, lưu trữ thành tích người chơi để tạo động lực cạnh tranh.
 - Thêm chế độ hướng dẫn chi tiết (tutorial) giúp người chơi mới làm quen nhanh hơn.
- **Ứng dụng công nghệ hiện đại**
 - Tích hợp công nghệ trí tuệ nhân tạo (AI) để phân tích và đưa ra gợi ý cho người chơi trong thời gian thực.
 - Sử dụng công nghệ đồ họa 3D hoặc 2.5D để tăng tính hấp dẫn cho trò chơi.
- **Xây dựng cộng đồng người dùng**
 - Xây dựng fanpage hoặc nhóm cộng đồng trên các nền tảng mạng xã hội để kết nối người chơi và nhận phản hồi từ họ.
 - Tổ chức các sự kiện, cuộc thi nhỏ liên quan đến game Tetris để quảng bá và thu hút người dùng mới.
 - **Đa nền tảng hóa**
 - Phát triển phiên bản game trên các nền tảng khác như Android, iOS hoặc thậm chí là trình duyệt web để tăng khả năng tiếp cận người chơi.

III. KẾT LUẬN

1. Kết luận

Dự án phát triển game Tetris đã mang đến cho nhóm những bài học quý giá, không chỉ về mặt kỹ thuật mà còn về cách thức triển khai một sản phẩm phần mềm từ ý tưởng đến thực tế. Mặc dù nhóm đã hoàn thành phiên bản Tetris cơ bản với các chức năng cần thiết, nhưng vẫn còn nhiều tính năng nâng cao chưa thể triển khai do hạn chế về thời gian và kỹ năng. Qua đó, nhóm nhận thấy tầm quan trọng của việc phân bổ thời gian hợp lý trong quá trình phát triển và cũng hiểu rõ hơn về các yếu tố cần thiết để cải thiện chất lượng sản phẩm. Bài học lớn nhất là cần tăng cường khả năng nghiên cứu và áp dụng công nghệ mới, từ đó có thể cải thiện hiệu quả công việc và tạo ra những sản phẩm phần mềm chất lượng hơn. Nhóm dự định sẽ tiếp tục phát triển game Tetris bằng cách tối ưu hóa hiệu suất, mở rộng tính năng, và nâng cao giao diện người dùng, đồng thời phát triển cộng đồng người chơi, đưa trò chơi đến gần hơn với nhiều đối tượng hơn.

2. Thành tựu

Trong quá trình thực hiện dự án, nhóm đã hoàn thành phiên bản Tetris cơ bản đáp ứng đầy đủ các yêu cầu về quản lý khối, tính điểm và xử lý logic trò chơi. Dù giao diện đơn giản nhưng đã được thiết kế sao cho trực quan và dễ sử dụng, mang lại trải nghiệm tốt cho người chơi ở nhiều độ tuổi và kỹ năng khác nhau. Ngoài ra, dự án còn giúp nhóm cải thiện đáng kể kỹ năng lập trình, cũng như khả năng làm việc nhóm và giải quyết các vấn đề phát sinh trong suốt quá trình phát triển. Những thành tựu này là kết quả của sự nỗ lực và hợp tác hiệu quả trong nhóm, đồng thời cũng là nền tảng vững chắc để nhóm có thể tiếp tục tham gia vào những dự án công nghệ phức tạp hơn trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Những quy tắc cơ bản để thiết kế một ứng dụng. Truy cập ngày 29/10/2024.

Tại link : <https://winds.vn/thiet-ke-giao-dien-app/>

[2] Cách cài đặt pygame và tạo mẫu để phát triển trò chơi bằng Python 3. Truy cập ngày

3/11/2024. Tại link: <https://galaxyz.net/cach-cai-dat-pygame-va-tao-mau-de-phat-trien-tro-choi-bang-python-3.2219.aneews>

[3] Các yếu tố lập trình game với Pygame. Truy cập ngày 4/11/2024. Tại link:

<https://codelearn.io/sharing/lap-trinh-game-co-ban-voi-pygame>

[4] PyGame: A Primer on Game Programming in Python. Truy cập ngày 5/11/2024. Tại

link: <https://realpython.com/pygame-a-primer/>